

## ACCELERATED FORWARD-BACKWARD OPTIMIZATION USING DEEP LEARNING\*

SEBASTIAN BANERT<sup>†</sup>, JEVGENIJA RUDZUSIKA<sup>‡</sup>, OZAN ÖKTEM<sup>‡</sup>, AND  
JONAS ADLER<sup>§</sup>

**Abstract.** We propose several deep-learning accelerated optimization solvers with convergence guarantees. We use ideas from the analysis of accelerated forward-backward schemes like FISTA, but instead of the classical approach of proving convergence for a choice of parameters, such as a step-size, we show convergence whenever the update is chosen in a specific set. Rather than picking a point in this set using some predefined method, we train a deep neural network to pick the best update within a given space. Finally, we show that the method is applicable to several cases of smooth and nonsmooth optimization and show superior results to established accelerated solvers.

**Key words.** convex optimization, deep learning, proximal-gradient algorithm, inverse problems

**MSC codes.** 90C25, 90C06, 68T07, 49M37, 65K10

**DOI.** 10.1137/22M1532548

**1. Introduction.** The tremendous growth in computing capacity of hardware and advances in numerical methods have allowed for very large-scale and detailed simulations of complex phenomena. Much focus is currently on embedding such simulation techniques into procedures for design, control, data assimilation, and inverse modeling (solving inverse problems/parameter estimation). These tasks are often formalized as continuous variable optimization problems. Approximation and discretization of continuous problems result in a very large problem size, both in the number of state variables and state equations, which in many scientific and industrial problems easily reaches the millions. Another further complicating factor is that the objective involved in the optimization may not necessarily be differentiable, like when sparsity is used in inverse modeling.

For the above-stated reasons, *solving large-scale nonsmooth optimization problems is one of the key challenges in scientific computing*. This has catalyzed a surge in research on methods that utilize specific structure in the problem and take advantage of modern multicore or distributed computing architectures. An example is decomposition algorithms that allow considerable computational speedup in parallel computing environments while addressing memory requirements. Another is operator splitting

---

\*Received by the editors November 3, 2022; accepted for publication (in revised form) November 24, 2023; published electronically April 1, 2024. Equal contribution by Sebastian Banert and Jevgenija Rudzusika.

<https://doi.org/10.1137/22M1532548>

**Funding:** The first author was supported by the Swedish Foundation of Strategic Research grant AM13-0049, by the Wallenberg AI, Autonomous Systems and Software Program, and by the Vetenskapsrådet grant 2021-05710. He is a member of the ELLIIT Strategic Research Area at Lund University. The second author was supported by the Swedish Foundation of Strategic Research grant AM13-0049, grant from the VINNOVA Open Innovation Hub project 2015-06759, and by Philips Healthcare. The fourth author was supported by the Swedish Foundation of Strategic Research grant AM13-0049, Industrial PhD grant ID14-0055, and by Elekta Instrument AB. Finally, the third author was supported by the Swedish Foundation of Strategic Research grant AM13-0049.

<sup>†</sup>Department of Automatic Control, Lund University, Lund, 223 63, Sweden (sebastian.banert@control.lth.se).

<sup>‡</sup>Department of Mathematics, KTH Royal Institute of Technology, Stockholm, 10044, Sweden (jevaks@kth.se, ozan@kth.se).

<sup>§</sup>DeepMind, London, United Kingdom (jonasadler@google.com).

techniques, which are designed to solve optimization problems, where the objective is a sum of two convex functions, one differentiable with a Lipschitz continuous gradient and the other possibly nonsmooth but with accessible proximal points. The latest line of development seeks to further improve upon computational feasibility by combining the aforementioned techniques from convex analysis with contemporary techniques from deep learning.

**1.1. Mathematical setting.** Let  $\mathcal{X}$  denote a Hilbert space with inner product  $\langle \cdot, \cdot \rangle$  and corresponding norm  $\|\cdot\|$ . Next, consider minimizing a differentiable convex function  $f: \mathcal{X} \rightarrow \mathbb{R}$  whose gradient  $\nabla f: \mathcal{X} \rightarrow \mathcal{X}$  is Lipschitz continuous with constant  $\beta^{-1} > 0$ . This can be solved by gradient descent [17] or accelerated methods based on an overshooting idea formulated by Nesterov [36].

The objective in the aforementioned minimization is smooth, but as already mentioned, many applications involve minimizing a nonsmooth objective function. To handle the latter, there has been significant effort to develop theory and algorithms suitable for nonsmooth optimization. These often utilize further structure in the objective. In particular, *splitting* techniques have been developed to solve optimization problems where the (nonsmooth) objective function is a sum of two convex functions, one smooth and the other nonsmooth. Stated mathematically, this refers to optimization problems of the form

$$(1.1) \quad \min_{x \in \mathcal{X}} f(x) + g(x),$$

where  $f: \mathcal{X} \rightarrow \mathbb{R}$  is a differentiable convex function and  $g: \mathcal{X} \rightarrow \overline{\mathbb{R}}$  is a proper, convex, and lower semicontinuous, but possibly nonsmooth function. Algorithms to solve (1.1) include the forward-backward splitting method [24] and its accelerated variants like fast ISTA (FISTA) [11]. Much attention has also been directed towards approaches that use learning techniques to improve the speed, like in learned ISTA (LISTA) [26]; see the survey [18] and references therein. However, the latter methods lack convergence guarantees. Another approach is to learn the parameters of a general optimization solver [9], but in this case there is a relatively small number of free parameters, which limits the potential of the scheme to be adapted to a particular problem class. To address this and increase the number of degrees of freedom in the learning, we will in this paper consider a *deviation-based* approach. Instead of learning parameters of a solver, we learn an entire updating function and then use that in an optimization solver. To ensure convergence, we constrain the updating function to sufficiently small deviations from known standard algorithms.

**1.2. Related work.** The overall principle that underlies application of machine learning for solving optimization problems is to use principles from statistical decision theory to select the “best” solver by training against data. Solvers are parametrized and “best” means selecting the solver parameters that lead to the smallest average error when applied to a collection of similar optimization problems.

Several authors have investigated the idea of deriving optimization schemes from statistical learning. One of the most notable schemes is LISTA introduced in [26]. This scheme was inspired by the Iterative Shrinkage-Thresholding Algorithm (ISTA), which is a popular algorithm for solving the optimization in sparse coding. Each iteration of ISTA is a combination of matrix operations followed by a thresholding function. Thus, when stopped after fixed number of iterations, the algorithm resembles a forward pass through a neural network. The idea in LISTA is to replace the handcrafted matrices in ISTA with learned ones that are trained in a supervised manner, provided that the

ground truth solution to the optimization problem is available. In contrast, we follow [9] and train the parameters of our optimization scheme using an unsupervised loss that is equal to the value of the objective function attained after a fixed number of iterations.

The above idea of learning a truncated optimization scheme in an unsupervised manner is also studied in machine learning [7, 30, 44, 12, 33]. The main use case considered in these works is the problem of training a neural network, which itself is a non-convex optimization problem. Here the approach is to replace conventional optimizers with learned optimizers that are faster/better. This is achieved by parametrizing updates at each iteration with neural networks and training the associated parameters either by variants of stochastic gradient descent [44, 7], by reinforcement learning [30, 12], or by using evolutionary strategies [35]. The challenge is to ensure that the learned optimizer generalizes from one optimization task to another. A difficulty here is to identify a class of optimization problems that share similar structure. We instead consider (large) classes of optimization problems that *naturally* share similarities, like those in image reconstruction for computed tomography (CT) and regularization of other inverse problems. This makes it easier to specify the generalization requirement.

Convergence of the above-mentioned learned optimization schemes is not guaranteed. Instead, their usage relies on (very strong) heuristics, so these data-driven approaches cannot be seen as optimization solvers in the classical sense. As we consider convex problems, our approach is not applicable for training neural networks, but we can on the other hand provide convergence guarantees.

One way to ensure convergence of learned optimization schemes is to apply them together with a “safe-guarding” mechanism [32, 28]. In the most simple case, at each iteration one can compare updates proposed by a learned optimization scheme with updates of a traditional convergent scheme, like ISTA, and choose the update that minimizes the objective [32]. However, it is well known that some schemes can lead to faster convergence while not being monotonic—for instance, schemes that gain acceleration through introducing momentum, like FISTA. Also, learned optimization algorithms might not exhibit monotonicity. Thus, more elaborate safeguarding principles described in [28] measure the progress of optimization by considering more than one previous iteration.

Another alternative is to investigate optimal parameter choices in schemes like FISTA when iterations are truncated; see, e.g., [31] and notably [19], which proves linear convergence of a related scheme under some further assumptions (sparsity, finite dimension, optimal training). Another approach is [9], which proves convergence of a learned primal-dual scheme under weaker assumptions. The analysis is however only performed in the parametric case, e.g., with an explicit form of the updating operation.

Many authors have proposed solving challenging inverse problems using neural network architectures inspired by existing optimization schemes. Even though such architectures are obtained by unrolling an optimization solver, they are typically trained in a supervised manner [3, 4, 27, 45]. Therefore, these networks are not themselves optimization solvers; see [8, section 5] for further details. Another example from scientific computing is [29], which deals with solving partial differential equations. Hence, unrolling should primarily be seen as a way to assemble an appropriately domain-adapted deep neural network architecture [8, section 4.9.1].

Finally, an attempt to analyze and interpret learned optimizers by considering linear approximations of optimizers close to the stationary point is presented in [33].

### 1.3. Contributions.

- We present algorithms to solve convex optimization problems that allow for design parameters (deviations) that have the same dimensions as the variables of the problem instead of restricting ourselves to real numbers.
- The difference from other methods that allow for errors in the evaluation is that we later reuse the deviations, so they have to be known to the algorithm.
- Instead of a summability condition, we provide a per-iteration condition on the norm of deviations that ensures convergence properties. This is in contrast to summability conditions (see, e.g., [14, 20, 43]) that seem numerically difficult to enforce, in particular within a deep neural network architecture for learned optimization.
- The deviations allow for designing various momentum methods as long as the momentum terms are scaled to fit the condition on the norm of the deviations.
- We present a combination of the deviation-based method with deep learning, where the deviations are represented by neural networks that always satisfy the condition on the norm of the deviations.
- Numerical experiments show that the so-obtained deep learning models can outperform both classical accelerated optimization methods and analogous deep learning methods without convergence guarantees.

**1.4. Overview of the paper.** Section 2 recalls the necessary notions from convex optimization. This is followed by subsection 3.1, which establishes a worst-case convergence result for a nonparametric gradient descent scheme for smooth optimization problems. Next, subsection 3.2 establishes a slightly weaker convergence theorem for a nonparametric forward-backward scheme applicable for nonsmooth problems. Section 4 discusses the details of learning a nonparametric optimization scheme. Section 5 concludes with numerical experiments from an imaging application.

**2. Background from optimization theory.** The aim here is to introduce the notation and basic notions from optimization theory that are used throughout the paper. The main reference is [10], but the reader may also consult [37, 39] for related results.

A function  $f: \mathcal{X} \rightarrow \overline{\mathbb{R}} = \mathbb{R} \cup \{\pm\infty\}$  is *proper* if its domain is nonempty and it does not take the value  $-\infty$ , i.e., if

$$\text{dom } f := \{x \in \mathcal{X} \mid f(x) < +\infty\} \neq \emptyset \quad \text{and} \quad f(x) > -\infty \text{ for all } x \in \mathcal{X}.$$

It is *convex* if  $f((1-\lambda)x + \lambda y) \leq (1-\lambda)f(x) + \lambda f(y)$  holds for all  $x, y \in \mathcal{X}$  and  $0 \leq \lambda \leq 1$  and *strictly convex* if this inequality is strict for  $x \neq y$  and  $0 < \lambda < 1$ . A strictly convex function has at most one minimizer [10, Corollary 11.9]. Next, the *subdifferential* of a convex function  $f: \mathcal{X} \rightarrow \overline{\mathbb{R}}$  is a (set-valued) mapping  $\partial f: \mathcal{X} \rightarrow 2^{\mathcal{X}}$  defined as

$$\partial f(x) := \begin{cases} \{y \in \mathcal{X} \mid \forall z \in \mathcal{X} : f(z) \geq f(x) + \langle y, z - x \rangle\} & \text{if } f(x) \in \mathbb{R}, \\ \emptyset & \text{otherwise.} \end{cases}$$

If  $f$  is Fréchet differentiable at  $x$ , then  $\partial f(x)$  consists of a single element, namely the gradient of  $f$  at  $x$  [10, Proposition 17.31], i.e.,  $\partial f(x) = \{\nabla f(x)\}$ . Finally, the *proximal point* of a proper, convex, and lower semicontinuous function  $f: \mathcal{X} \rightarrow \overline{\mathbb{R}}$  at  $x \in \mathcal{X}$  with *step-size*  $\gamma > 0$  is defined as

$$\text{Prox}_f^\gamma(x) := \arg \min \left\{ f(z) + \frac{1}{2\gamma} \|z - x\|^2 \mid z \in \mathcal{X} \right\}.$$

The proximal point can be used to characterize points in the subdifferential [10, Proposition 16.44]:

$$p = \text{Prox}_f^\gamma(x) \iff \frac{1}{\gamma}(x - p) \in \partial f(p).$$

The operator  $\text{Prox}_f^\gamma$  is then a *firmly nonexpansive* single-valued mapping from  $\mathcal{X}$  to  $\mathcal{X}$  [10, Proposition 12.28],

$$\begin{aligned} & \left\| \text{Prox}_f^\gamma(x_1) - \text{Prox}_f^\gamma(x_2) \right\|^2 \\ & \leq \left\langle \text{Prox}_f^\gamma(x_1) - \text{Prox}_f^\gamma(x_2), x_1 - x_2 \right\rangle \text{ for all } x_1, x_2 \in \mathcal{X}. \end{aligned}$$

Furthermore,  $p = \text{Prox}_f^\gamma(x)$  if and only if the following variational inequality holds [10, Proposition 12.26]:

$$(2.1) \quad f(z) \geq f(p) + \frac{1}{\gamma} \langle x - p, z - p \rangle \quad \text{for all } z \in \mathcal{X}.$$

If  $\nabla f$  is Lipschitz continuous with constant  $\beta^{-1}$ , then the following inequality, which is sometimes called the *descent lemma* (see [10, Theorem 18.15]), holds:

$$(2.2) \quad f(x + d) \leq f(x) + \langle \nabla f(x), d \rangle + \frac{1}{2\beta} \|d\|^2, \quad x, d \in \mathcal{X}.$$

Likewise, if  $f$  is in addition assumed to be convex [37, Theorem 2.1.5],

$$(2.3) \quad f(x) + \langle \nabla f(x), y - x \rangle + \frac{\beta}{2} \|\nabla f(x) - \nabla f(y)\|^2 \leq f(y) \quad \text{for all } x, y \in \mathcal{X}.$$

**3. Deviation-based optimization schemes.** We call an optimization scheme *parameter-based* if convergence criteria are expressed as constraints on its parameters. An example of a parameter-based optimization scheme is the classical gradient descent method for minimizing a convex differentiable function  $f: \mathcal{X} \rightarrow \mathbb{R}$  where  $\nabla f: \mathcal{X} \rightarrow \mathcal{X}$  is  $\beta^{-1}$ -Lipschitz. To see this, note that such a method relies on an updating scheme of the following form:

$$(3.1) \quad x_{n+1} := x_n - \beta(1 + t_n)\nabla f(x_n).$$

For convergence to a minimizer for  $f$ ,  $t_n$  is a parameter that needs to satisfy  $-1 < t_n < 1$  and  $\sum_{n=0}^{\infty} (1 + t_n)(1 - t_n) = +\infty$  [10, Proposition 4.39, Corollary 5.16, and Corollary 18.17]. Thus, a convergence criterion for the above scheme is expressible in terms of a specific choice of parameters, so we are dealing with a parameter-based optimization method.

In fact, almost all methods in classical optimization theory are written as schemes of this form. Furthermore, it has been considered a virtue to have as few parameters as possible (preferably none). On the other hand, recent advances in machine learning (in particular, availability of large data sets and computational resources) allow us to deal with extremely large numbers of parameters while avoiding over-fitting effects and achieve significant improvements over established methods. While optimization schemes can be formulated using neural networks, and their convergence can be studied in terms of their parameters, this is typically very cumbersome and restricts neural network architectures to, e.g., one layer networks [9].

To allow for a wider range of neural network architectures, we need to move away from convergence guarantees that involve the parameters of the scheme. The idea is to give the network as much freedom as possible while still retaining the stability properties of the underlying optimization method. Such schemes that do not impose an a priori restriction on the number of parameters will be referred to as *deviation-based*.

To introduce a deviation-based scheme for smooth optimization as in the setting above, we modify the gradient descent method in (3.1) by introducing a sequence  $(\Delta x_n)_{n \geq 0}$  of deviations:

$$(3.2) \quad x_{n+1} := x_n - \beta(\nabla f(x_n) + \Delta x_n) \quad \text{for some deviation/offset } \Delta x_n \in \mathcal{X}.$$

The classical convergence proof for the gradient descent implies convergence if  $\Delta x_n = t \nabla f(x_n)$  for  $-1 < t < 1$ . This is, however, a one-dimensional set and hardly interesting to do learning on, and in particular not deep learning. In what follows, we consider schemes of the above form and prove convergence for sets of updates  $\Delta x_n \in \mathcal{X}$  that have full dimension, thus allowing for a much greater degree of freedom in selecting the updates. The learning itself will be discussed in section 4.

**3.1. Deviation-based optimization schemes for smooth optimization.** In previous literature, the sequence  $(\Delta x_n)_{n \geq 0}$  of deviations in (3.2) was used to model errors in the evaluation of the gradient; see, for example, [14, 20, 43]. In our setting, however, we are interested in consciously choosing  $\Delta x_n \in \mathcal{X}$  of (3.1) which will preserve or, by a clever choice, might improve its convergence properties. In these papers, convergence is ensured by a summability criterion. In our work, the summability criterion is replaced by condition (3.4) that is implementable by-iteration.

As we show below, the condition  $\|\Delta x_n\| \leq \|\nabla f(x_n)\|$  guarantees that the function value  $f(x_n)$  is nonincreasing, and it is the weakest condition which guarantees this property. In particular, we will show that the function values  $f(x_n)$  converge to the minimal value of the function  $f$ . Special cases of the following theorem are discussed in Remark 3.2 below.

**THEOREM 3.1.** *Consider minimizing  $f: \mathcal{X} \rightarrow \mathbb{R}$  that is convex and differentiable with a gradient that is  $\beta^{-1}$ -Lipschitz continuous for some  $\beta > 0$ . Next, consider the iterative scheme*

$$(3.3) \quad x_{n+1} := x_n - \beta(\nabla f(x_n) + \Delta x_n) \quad \text{for all } n \geq 0,$$

where  $x_0 \in \mathcal{X}$  is some fixed initial point and  $(\Delta x_n)_{n \geq 0}$  is any sequence in  $\mathcal{X}$  such that

$$(3.4) \quad \|\Delta x_n\| \leq \alpha_n \|\nabla f(x_n)\|$$

for some sequence  $(\alpha_n)_{n \geq 0}$  of positive numbers with  $\alpha_n \leq 1$  for all  $n \geq 0$ .

1. Then  $(f(x_n))_{n \geq 0}$  is a nonincreasing sequence of objective function values.
2. The following inequality holds:

$$(3.5) \quad f(x_n) - f(\bar{x}) \leq \frac{1}{2\beta} \left( \prod_{k=0}^{n-1} \left( 1 - \frac{1 - \alpha_k^2}{k+2} \right) \right) \|x_0 - \bar{x}\|^2 \quad \text{for all } n \geq 0,$$

where  $\bar{x}$  denotes a global minimizer of  $f$ .

3. If we further assume that

$$(3.6) \quad \sum_{n=0}^{\infty} \frac{1 - \alpha_n}{n+2} = +\infty,$$

then  $f(x_n) \rightarrow f(\bar{x})$  as  $n \rightarrow +\infty$  with  $\bar{x}$  denoting a global minimizer of  $f$ .

*Remark 3.2.* Choosing  $\alpha_n = \alpha$  for all  $n$  in Theorem 3.1 simplifies (3.5) to

$$(3.7) \quad \begin{aligned} f(x_n) - f(\bar{x}) &\leq \frac{1}{2\beta} \left( \prod_{k=0}^{n-1} \left( \frac{k+1+\alpha^2}{k+2} \right) \right) \|x_0 - \bar{x}\|^2 \\ &= \frac{\Gamma(n+\alpha^2+1)}{2\beta\Gamma(n+2)\Gamma(1+\alpha^2)} \|x_0 - \bar{x}\|^2. \end{aligned}$$

Hence, using Stirling's formula, we obtain

$$\frac{f(x_n) - f(\bar{x})}{\|x_0 - \bar{x}\|^2} = O\left(n^{-(1-\alpha^2)}\right) \quad \text{as } n \rightarrow \infty.$$

When  $\alpha = 0$ , i.e., if  $\Delta x_n = 0$  for  $n = 0, 1, \dots$ , we get

$$f(x_n) - f(\bar{x}) = \frac{1}{2\beta(n+1)} \|x_0 - \bar{x}\|^2.$$

Hence, we have (asymptotically) recovered the optimal rate from [22, Theorem 1] up to a constant of 2.

Note also that all the calculations work for a smaller  $0 < \beta' < \beta$  since  $1/\beta'$  will also be a Lipschitz constant of  $\nabla f$ . In addition to the convergence of the function values, we are also able to show the convergence of the iteration sequence with stronger assumptions on the objective function  $f$ .

**COROLLARY 3.3.** *If, in addition to the assumptions in item 3 of Theorem 3.1, the objective function  $f$  is coercive, i.e.,  $f(x) \rightarrow +\infty$  as  $\|x\| \rightarrow +\infty$ , and strictly convex, then the sequence  $(x_n)_{n \geq 0}$  converges weakly to the unique minimizer of  $f$ .*

*Proof.* The sequence  $(x_n)_{n \geq 0}$  is bounded: by item 1 of Theorem 3.1, the sequence  $(f(x_n))_{n \geq 0}$  is bounded from above by  $f(x_0) \in \mathbb{R}$ . The asserted boundedness then follows from the coercivity of  $f$ .

If  $x \in \mathcal{X}$  is a weak sequential cluster point, i.e., there exists a subsequence  $(x_{n_k})_{k \geq 0}$  of  $(x_n)_{n \geq 0}$  with  $x_{n_k} \rightharpoonup x$ , then [10, Theorem 9.1] and item 3 of Theorem 3.1 give  $f(x) \leq \liminf_{n \rightarrow \infty} f(x_{n_k}) = \inf_{x \in \mathcal{X}} f(x)$ , i.e.,  $x$  is the unique minimizer of  $f$ . The statement of the corollary then follows from [10, Lemma 2.46].  $\square$

*Proof of Theorem 3.1.* For all  $n \geq 0$  we have, by (2.3),

$$(3.8) \quad \begin{aligned} f(x_{n+1}) - f(x_n) &\leq \langle \nabla f(x_{n+1}), x_{n+1} - x_n \rangle - \frac{\beta}{2} \|\nabla f(x_{n+1}) - \nabla f(x_n)\|^2 \\ &= -\beta \langle \nabla f(x_{n+1}), \Delta x_n \rangle - \frac{\beta}{2} \|\nabla f(x_{n+1})\|^2 - \frac{\beta}{2} \|\nabla f(x_n)\|^2 \\ &= -\frac{\beta}{2} \|\nabla f(x_{n+1}) + \Delta x_n\|^2 + \frac{\beta}{2} (\|\Delta x_n\|^2 - \|\nabla f(x_n)\|^2). \end{aligned}$$

The proof of item 1 now follows from the observation that

$$\|\Delta x_n\|^2 - \|\nabla f(x_n)\|^2 \leq -(1 - \alpha_n^2) \|\nabla f(x_n)\|^2.$$

We will next prove the formula in (3.5) by proving the following stronger relation:

$$(3.9) \quad \begin{aligned} f(x_n) - f(\bar{x}) &\leq \frac{1}{2\beta} \left( \prod_{k=0}^{n-1} \left( 1 - \frac{1 - \alpha_k^2}{k+2} \right) \right) \|x_0 - \bar{x}\|^2 \\ &\quad - \frac{1}{2\beta(n+1)} \|x_n - \bar{x} - \beta \nabla f(x_n)\|^2, \end{aligned}$$

where  $\bar{x} \in \mathcal{X}$  denotes a global minimizer of  $f$ , i.e.,  $\nabla f(\bar{x}) = 0$ . To prove the above, we proceed by induction and begin by considering the case  $n = 0$ . By (2.3), we have

$$\begin{aligned} f(x_0) - f(\bar{x}) &\leq \langle \nabla f(x_0), x_0 - \bar{x} \rangle - \frac{\beta}{2} \|\nabla f(x_0)\|^2 \\ &= \frac{1}{2\beta} \|x_0 - \bar{x}\|^2 - \frac{1}{2\beta} \|x_0 - \bar{x} - \beta \nabla f(x_0)\|^2. \end{aligned}$$

This proves the claim for  $n = 0$ , i.e., the induction basis holds. Next, assume (3.9) holds for some  $n \geq 0$  and we seek to prove that it holds for  $n + 1$ . Then

$$\begin{aligned} f(x_{n+1}) - f(\bar{x}) &= \frac{1 - \alpha_n^2}{n+2} (f(x_{n+1}) - f(\bar{x})) \\ &\quad + \left(1 - \frac{1 - \alpha_n^2}{n+2}\right) (f(x_{n+1}) - f(x_n)) + \left(1 - \frac{1 - \alpha_n^2}{n+2}\right) (f(x_n) - f(\bar{x})). \end{aligned}$$

Here, we estimate the first two terms with (2.3) and the last one with the induction hypothesis (3.9). This results in

$$\begin{aligned} f(x_{n+1}) - f(\bar{x}) &\leq \frac{1 - \alpha_n^2}{n+2} \left( \langle \nabla f(x_{n+1}), x_{n+1} - \bar{x} \rangle - \frac{\beta}{2} \|\nabla f(x_{n+1})\|^2 \right) \\ &\quad + \left(1 - \frac{1 - \alpha_n^2}{n+2}\right) \left( \langle \nabla f(x_{n+1}), x_{n+1} - x_n \rangle - \frac{\beta}{2} \|\nabla f(x_{n+1}) - \nabla f(x_n)\|^2 \right) \\ &\quad + \frac{1}{2\beta} \left( \prod_{k=0}^n \left(1 - \frac{1 - \alpha_k^2}{k+2}\right) \right) \|x_0 - \bar{x}\|^2 \\ &\quad - \left(1 - \frac{1 - \alpha_n^2}{n+2}\right) \frac{1}{2\beta(n+1)} \|x_n - \bar{x} - \beta \nabla f(x_n)\|^2. \end{aligned}$$

Using (3.3) and reordering the terms gives

$$\begin{aligned} (3.10) \quad f(x_{n+1}) - f(\bar{x}) &\leq -\beta \langle \nabla f(x_{n+1}), \Delta x_n \rangle - \frac{\beta}{2} \|\nabla f(x_{n+1})\|^2 \\ &\quad + \frac{1 - \alpha_n^2}{n+2} \langle \nabla f(x_{n+1}), x_n - \bar{x} - \beta \nabla f(x_n) \rangle \\ &\quad - \left(1 - \frac{1 - \alpha_n^2}{n+2}\right) \left( \frac{\beta}{2} \|\nabla f(x_n)\|^2 + \frac{1}{2\beta(n+1)} \|x_n - \bar{x} - \beta \nabla f(x_n)\|^2 \right) \\ &\quad + \frac{1}{2\beta} \left( \prod_{k=0}^n \left(1 - \frac{1 - \alpha_k^2}{k+2}\right) \right) \|x_0 - \bar{x}\|^2. \end{aligned}$$

On the other hand, using again (3.3),

$$\begin{aligned} &\frac{1}{2\beta(n+2)} \|x_{n+1} - \bar{x} - \beta \nabla f(x_{n+1})\|^2 \\ &= \frac{1}{2\beta(n+2)} \|x_n - \bar{x} - \beta \nabla f(x_n)\|^2 + \frac{\beta}{2(n+2)} \|\Delta x_n + \nabla f(x_{n+1})\|^2 \\ &\quad - \frac{1}{n+2} \langle x_n - \bar{x} - \beta \nabla f(x_n), \Delta x_n + \nabla f(x_{n+1}) \rangle. \end{aligned}$$



Adding this to (3.10), followed by a simple calculation, yields

$$\begin{aligned} & f(x_{n+1}) - f(\bar{x}) + \frac{1}{2\beta(n+2)} \|x_{n+1} - \bar{x} - \beta \nabla f(x_{n+1})\|^2 \\ & \leq \frac{1}{2\beta} \left( \prod_{k=0}^n \left( 1 - \frac{1 - \alpha_k^2}{k+2} \right) \right) \|x_0 - \bar{x}\|^2 - \frac{\beta(n+1)}{2(n+2)} (1 - \alpha_n^2) \|\nabla f(x_{n+1})\|^2 \\ & \quad - \frac{\alpha_n^2}{2\beta(n+1)(n+2)} \left\| x_n - \bar{x} - \beta \nabla f(x_n) + \frac{\beta(n+1)}{\alpha_n^2} \Delta x_n + \beta(n+1) \nabla f(x_{n+1}) \right\|^2 \\ & \quad - \frac{\beta}{2} \left( 1 - \frac{1 - \alpha_n^2}{n+2} \right) \left( \|\nabla f(x_n)\|^2 - \frac{1}{\alpha_n^2} \|\Delta x_n\|^2 \right). \end{aligned}$$

By (3.4) and taking into account the nonnegativity of norms, we get

$$\begin{aligned} & f(x_{n+1}) - f(\bar{x}) + \frac{1}{2\beta(n+2)} \|x_{n+1} - \bar{x} - \beta \nabla f(x_{n+1})\|^2 \\ & \leq \frac{1}{2\beta} \left( \prod_{k=0}^n \left( 1 - \frac{1 - \alpha_k^2}{k+2} \right) \right) \|x_0 - \bar{x}\|^2, \end{aligned}$$

which finishes the induction. Let us now prove the convergence  $f(x_n) \rightarrow f(\bar{x})$  as  $n \rightarrow +\infty$ . Taking the logarithm in (3.5) gives

$$\log(f(x_n) - f(\bar{x})) \leq \sum_{k=0}^{n-1} \log \left( 1 - \frac{1 - \alpha_k^2}{k+2} \right) + \log \|x_0 - \bar{x}\|^2 - \log(2\beta).$$

Since  $\log$  is a concave function,  $\log(1 - x) \leq -x$  for all  $x \in \mathbb{R}$ . Hence,

$$\begin{aligned} \log(f(x_n) - f(\bar{x})) & \leq - \sum_{k=0}^{n-1} \frac{1 - \alpha_k^2}{k+2} + \log \|x_0 - \bar{x}\|^2 - \log(2\beta) \\ & \leq - \sum_{k=0}^{n-1} \frac{1 - \alpha_k}{k+2} + \log \|x_0 - \bar{x}\|^2 - \log(2\beta). \end{aligned}$$

Because of the assumption in (3.6), the right-hand side converges to  $-\infty$ , so  $f(x_n) \rightarrow f(\bar{x})$  as  $n \rightarrow \infty$ .  $\square$

**3.2. Deviation-based schemes for nonsmooth optimization.** The theory in subsection 3.1 only applies to optimization problems with a smooth objective. Theory for optimization with a nonsmooth objective needs to be based on subdifferential calculus. Our focus is on minimizing a nonsmooth objective with an additional structure as in (1.1).

The proposed deviation-based iterative scheme is inspired by the classical forward-backward (or proximal-gradient/ISTA) scheme [24], but with deviations in the spirit of the gradient method outlined in subsection 3.1. The next theorem states the scheme and analyzes its convergence properties for solving the nonsmooth minimization problem in (1.1) where  $g$  is not necessarily differentiable.

**THEOREM 3.4.** *Consider minimizing  $f + g$  where  $f : \mathcal{X} \rightarrow \mathbb{R}$  is convex and differentiable with a gradient  $\nabla f : \mathcal{X} \rightarrow \mathcal{X}$  that is Lipschitz continuous with constant  $\beta^{-1}$  for some  $\beta > 0$ , and  $g : \mathcal{X} \rightarrow \overline{\mathbb{R}}$  is a proper, convex, and lower semicontinuous function that is not necessarily differentiable. Next, consider the iterative scheme*

$$(3.11a) \quad w_n := x_n + \Delta x_n^1,$$

$$(3.11b) \quad x_{n+1} := \text{Prox}_g^{\gamma_n} \left( x_n - \gamma_n \nabla f(w_n) + \frac{\gamma_n}{\beta} \Delta x_n^1 + \Delta x_n^2 \right),$$

where  $x_0 \in \mathcal{X}$  is some fixed initial point,  $0 < \gamma_n < 2\beta$  for all  $n \geq 0$ , and

$$(3.12) \quad \begin{aligned} & \frac{1}{2\beta} \|\Delta x_n^1\|^2 + \frac{\beta}{2\gamma_n(2\beta - \gamma_n)} \|\Delta x_n^2\|^2 \\ & \leq \alpha_n^{(1)} \frac{2\beta - \gamma_{n-1}}{2\beta\gamma_{n-1}} \left\| x_n - x_{n-1} - \frac{\beta}{2\beta - \gamma_{n-1}} \Delta x_{n-1}^2 \right\|^2 \\ & \quad + \alpha_n^{(2)} \frac{\beta}{2} \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta} (x_n - w_{n-1}) \right\|^2 \end{aligned}$$

holds for all  $n \geq 1$ . Then, the following hold:

1. For  $n \geq 0$ , define

$$V_n := f(w_n) + g(x_{n+1}) + \langle \nabla f(w_n), x_{n+1} - w_n \rangle + \frac{1}{2\beta} \|x_{n+1} - w_n\|^2.$$

Then  $V_n \geq f(x_{n+1}) + g(x_{n+1})$  for all  $n \geq 0$  and the sequence

$$(3.13) \quad \left( V_n + \frac{2\beta - \gamma_n}{2\beta\gamma_n} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 \right)_{n \geq 0}$$

is monotonically nonincreasing if  $\alpha_n^{(1)} \leq 1$  and  $\alpha_n^{(2)} \leq 1$  for all  $n \geq 0$ .

2. If  $f + g$  is bounded from below and

$$(3.14) \quad 0 < \liminf_{n \rightarrow \infty} \gamma_n \leq \limsup_{n \rightarrow \infty} \gamma_n < 2\beta,$$

$$(3.15) \quad 0 \leq \limsup_{n \rightarrow \infty} \alpha_n^{(1)} < 1, \quad \text{and} \quad 0 \leq \limsup_{n \rightarrow \infty} \alpha_n^{(2)} < 1$$

hold, then

$$\begin{aligned} \sum_{n=0}^{\infty} \|x_n - x_{n+1}\|^2 &< +\infty, & \sum_{n=0}^{\infty} \|\Delta x_n^1\|^2 &< +\infty, \\ \text{and} & & \sum_{n=0}^{\infty} \|\Delta x_n^2\|^2 &< +\infty. \end{aligned}$$

3. Under the conditions of item 2, any weak sequential cluster point of  $(x_n)_{n \geq 0}$  is a minimizer of  $f + g$ . If there are no such minimizers, then  $\|x_n\| \rightarrow +\infty$  as  $n \rightarrow \infty$ . Conversely, a weak sequential cluster point exists whenever the sequence  $(x_n)_{n \geq 0}$  is bounded.

**COROLLARY 3.5.** *If, in addition to the assumptions in item 3 of Theorem 3.4, the objective function  $f + g$  is coercive, i.e.,  $f(x) + g(x) \rightarrow +\infty$  as  $\|x\| \rightarrow +\infty$ , and strictly convex, then the sequence  $(x_n)_{n \geq 0}$  converges weakly to the unique minimizer of  $f + g$ .*

*Proof.* The sequence  $(x_n)_{n \geq 0}$  is bounded: by item 1 of Theorem 3.4, the nonincreasing real sequence in (3.20) guarantees that  $(V_n)_{n \geq 0}$  is bounded from above and

is greater than or equal to  $(f(x_{n+1}) + g(x_{n+1}))_{n \geq 0}$ . The asserted boundedness then follows from the coercivity of  $f + g$ .

The statement of the corollary then follows from [10, Lemma 2.46], taking into account item 3 of Theorem 3.4 and that the strict convexity of  $f + g$  ensures the uniqueness of the minimizer of  $f + g$ .  $\square$

*Proof of Theorem 3.4.* We start by proving the claims in item 1. The first inequality follows from (2.2), so we focus on proving the monotonic decrease of the sequence in (3.13). Note first that, for  $n \geq 0$ , (3.11b) is equivalent to

$$(3.16) \quad \frac{x_n - x_{n+1}}{\gamma_n} - \nabla f(w_n) + \frac{1}{\beta} \Delta x_n^1 + \frac{1}{\gamma_n} \Delta x_n^2 \in \partial g(x_{n+1}).$$

This yields

$$(3.17) \quad \begin{aligned} g(x_n) &\geq g(x_{n+1}) + \frac{1}{\gamma_n} \|x_n - x_{n+1}\|^2 - \langle \nabla f(w_n), x_n - x_{n+1} \rangle \\ &\quad + \frac{1}{\beta} \langle \Delta x_n^1, x_n - x_{n+1} \rangle + \frac{1}{\gamma_n} \langle \Delta x_n^2, x_n - x_{n+1} \rangle. \end{aligned}$$

By (2.3) we get the following inequality for all  $n \geq 1$ :

$$(3.18) \quad f(w_{n-1}) \geq f(w_n) + \langle \nabla f(w_n), w_{n-1} - w_n \rangle + \frac{\beta}{2} \|\nabla f(w_n) - \nabla f(w_{n-1})\|^2.$$

From (3.18) and (3.17) and the definition of  $V_n$ , we therefore obtain

$$(3.19) \quad \begin{aligned} V_{n-1} - V_n &= f(w_{n-1}) - f(w_n) + g(x_n) - g(x_{n+1}) + \langle \nabla f(w_{n-1}), x_n - w_{n-1} \rangle \\ &\quad + \frac{1}{2\beta} \|x_n - w_{n-1}\|^2 - \langle \nabla f(w_n), x_{n+1} - w_n \rangle - \frac{1}{2\beta} \|x_{n+1} - w_n\|^2 \\ &\geq \langle \nabla f(w_n) - \nabla f(w_{n-1}), w_{n-1} - x_n \rangle + \frac{\beta}{2} \|\nabla f(w_n) - \nabla f(w_{n-1})\|^2 \\ &\quad + \frac{1}{2\beta} \|x_n - w_{n-1}\|^2 + \frac{1}{\gamma_n} \|x_n - x_{n+1}\|^2 - \frac{1}{2\beta} \|x_{n+1} - w_n\|^2 \\ &\quad + \frac{1}{\beta} \langle \Delta x_n^1, x_n - x_{n+1} \rangle + \frac{1}{\gamma_n} \langle \Delta x_n^2, x_n - x_{n+1} \rangle \\ &= \frac{\beta}{2} \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta} (x_n - w_{n-1}) \right\|^2 \\ &\quad + \frac{2\beta - \gamma_n}{2\beta\gamma_n} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 \\ &\quad - \frac{1}{2\beta} \|\Delta x_n^1\|^2 - \frac{\beta}{2\gamma_n(2\beta - \gamma_n)} \|\Delta x_n^2\|^2. \end{aligned}$$

Using (3.12), we then obtain the following inequality for all  $n \geq 1$ :

$$(3.20) \quad \begin{aligned} V_{n-1} &+ \frac{2\beta - \gamma_{n-1}}{2\beta\gamma_{n-1}} \left\| x_n - x_{n-1} - \frac{\beta}{2\beta - \gamma_{n-1}} \Delta x_{n-1}^2 \right\|^2 \\ &\geq V_n + \frac{2\beta - \gamma_n}{2\beta\gamma_n} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 \\ &\quad + \frac{\beta(1 - \alpha_n^{(2)})}{2} \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta} (x_n - w_{n-1}) \right\|^2 \\ &\quad + \frac{(1 - \alpha_n^{(1)})(2\beta - \gamma_{n-1})}{2\beta\gamma_{n-1}} \left\| x_n - x_{n-1} - \frac{\beta}{2\beta - \gamma_{n-1}} \Delta x_{n-1}^2 \right\|^2. \end{aligned}$$

The claim that the sequence in (3.13) is monotonically nonincreasing now follows from ignoring the two last terms in (3.20), which are nonnegative. This concludes the proof of the claims in item 1.

We next turn our attention to proving the claim in item 2. First, let  $M \in \mathbb{R}$  such that  $f(x) + g(x) \geq M > -\infty$  holds for all  $x \in \mathcal{X}$  as  $f + g$  is (by assumption) bounded from below. For fixed  $N \geq 1$ , consider the sum over  $n = 1, \dots, N$  for the terms in (3.20):

$$\begin{aligned} & V_0 + \frac{2\beta - \gamma_0}{2\beta\gamma_0} \left\| x_1 - x_0 - \frac{\beta}{2\beta - \gamma_0} \Delta x_0^2 \right\|^2 \\ & \geq V_N + \frac{2\beta - \gamma_N}{2\beta\gamma_N} \left\| x_{N+1} - x_N - \frac{\beta}{2\beta - \gamma_N} \Delta x_N^2 \right\|^2 \\ & \quad + \sum_{n=1}^N \frac{\beta(1 - \alpha_n^{(2)})}{2} \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta}(x_n - w_{n-1}) \right\|^2 \\ & \quad + \sum_{n=0}^{N-1} \frac{(1 - \alpha_{n+1}^{(1)})(2\beta - \gamma_n)}{2\beta\gamma_n} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2. \end{aligned}$$

By the claim in item 1, we know  $V_N \geq f(x_{N+1}) + g(x_{N+1}) \geq M$ , so

$$\begin{aligned} & \sum_{n=1}^N \frac{\beta(1 - \alpha_n^{(2)})}{2} \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta}(x_n - w_{n-1}) \right\|^2 \\ & \quad + \sum_{n=0}^{N-1} \frac{(1 - \alpha_{n+1}^{(1)})(2\beta - \gamma_n)}{2\beta\gamma_n} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 \\ & \leq V_0 + \frac{2\beta - \gamma_0}{2\beta\gamma_0} \left\| x_1 - x_0 - \frac{\beta}{2\beta - \gamma_0} \Delta x_0^2 \right\|^2 - M. \end{aligned}$$

Next, by (3.14) and (3.15), there exist some  $n_0 \geq 0$ ,  $0 < \gamma_- \leq \gamma_+ < 2\beta$ ,  $0 \leq \alpha_+^{(1)} < 1$ , and  $0 \leq \alpha_+^{(2)} < 1$  such that

$$\gamma_- \leq \gamma_n \leq \gamma_+, \quad 0 \leq \alpha_n^{(1)} \leq \alpha_+^{(1)}, \quad \text{and} \quad 0 \leq \alpha_n^{(2)} \leq \alpha_+^{(2)} \quad \text{for all } n \geq n_0.$$

We therefore get

$$\begin{aligned} & \sum_{n=n_0}^N \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta}(x_n - w_{n-1}) \right\|^2 \\ & \leq \frac{2}{\beta(1 - \alpha_+^{(2)})} \left( V_0 + \frac{2\beta - \gamma_0}{2\beta\gamma_0} \left\| x_1 - x_0 - \frac{\beta}{2\beta - \gamma_0} \Delta x_0^2 \right\|^2 - M \right) \end{aligned}$$

and

$$\begin{aligned} & \sum_{n=n_0}^{N-1} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 \\ & \leq \frac{2\beta\gamma_+}{(1 - \alpha_+^{(1)})(2\beta - \gamma_+)} \left( V_0 + \frac{2\beta - \gamma_0}{2\beta\gamma_0} \left\| x_1 - x_0 - \frac{\beta}{2\beta - \gamma_0} \Delta x_0^2 \right\|^2 - M \right). \end{aligned}$$

Since the right-hand side is finite and independent of  $N$  in both cases, we get

$$\sum_{n=n_0}^{\infty} \left\| \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta}(x_n - w_{n-1}) \right\|^2 < +\infty,$$

$$\sum_{n=n_0}^{\infty} \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 < +\infty.$$

Using (3.12) and the fact that the convergence of the series does not depend on the (finite) number of elements with indices below  $n_0$ , we can immediately conclude that

$$\sum_{n=0}^{\infty} \|\Delta x_n^1\|^2 < +\infty \quad \text{and} \quad \sum_{n=0}^{\infty} \|\Delta x_n^2\|^2 < +\infty.$$

The last assertion of item 2 follows from the observation below, which also concludes the proof of the claims in item 2.

$$\sum_{n=0}^{\infty} \|x_{n+1} - x_n\|^2$$

$$\leq 2 \sum_{n=0}^{\infty} \left( \left\| x_{n+1} - x_n - \frac{\beta}{2\beta - \gamma_n} \Delta x_n^2 \right\|^2 + \frac{\beta^2}{(2\beta - \gamma_n)^2} \|\Delta x_n^2\|^2 \right) < \infty.$$

We finally prove the claim in item 3. Let  $x$  be a weak sequential cluster point of the sequence  $(x_n)_{n \geq 0}$  with a subsequence  $(n_k)_{k \geq 0}$  such that  $x_{n_k} \rightharpoonup x$  as  $k \rightarrow \infty$ . By (3.16), we have

$$(3.21) \quad \frac{x_{n_k} - x_{n_k+1}}{\gamma_{n_k}} + \nabla f(x_{n_k+1}) - \nabla f(w_{n_k}) + \frac{\Delta x_{n_k}^1}{\beta} + \frac{\Delta x_{n_k}^2}{\gamma_{n_k}} \in (\nabla f + \partial g)(x_{n_k+1})$$

for all  $k \geq 0$ . The assertion in item 2 combined with the Lipschitz continuity of  $\nabla f$  yields the following as  $k \rightarrow \infty$ :

$$0 \leq \left\| \frac{x_{n_k} - x_{n_k+1}}{\gamma_{n_k}} \right\| \leq \frac{\|x_{n_k} - x_{n_k+1}\|}{\gamma_-} \rightarrow 0,$$

$$0 \leq \|\nabla f(x_{n_k+1}) - \nabla f(w_{n_k})\| \leq \frac{1}{\beta} (\|x_{n_k+1} - x_{n_k}\| + \|\Delta x_{n_k}^1\|) \rightarrow 0,$$

$$0 \leq \left\| \frac{\Delta x_{n_k}^2}{\gamma_{n_k}} \right\| \leq \frac{1}{\gamma_-} \|\Delta x_{n_k}^2\| \rightarrow 0.$$

Hence, the left-hand side of (3.21) converges to 0 as  $k \rightarrow \infty$ . Thus, by [10, Proposition 20.38] we get  $x_{n_k+1} = (x_{n_k+1} - x_{n_k}) + x_{n_k} \rightharpoonup x$ ,  $0 \in (\nabla f + \partial g)(x)$ , i.e.,  $x$  is a minimizer of  $f + g$ . The last statement follows from the fact that if  $\|x_n\| \not\rightarrow +\infty$ , then there exists a bounded subsequence that has a weakly convergent subsequence whose weak limit is a minimizer of  $f + g$ . This concludes the proof of the claim in item 3.  $\square$

*Remark 3.6.* If  $g = 0$  on  $\mathcal{X}$ , then (3.11) reads as

$$w_{n+1} = w_n - \gamma_n \nabla f(w_n) + \Delta x_{n+1}^1 + \frac{\gamma_n - \beta}{\beta} \Delta x_n^1 + \Delta x_n^2,$$

and the condition in (3.12) becomes

(3.22)

$$\begin{aligned} & \frac{1}{2\beta} \|\Delta x_n^1\|^2 + \frac{\beta}{2\gamma_n(2\beta - \gamma_n)} \|\Delta x_n^2\|^2 \\ & \leq \frac{\alpha_n^{(1)}(2\beta - \gamma_{n-1})}{2\beta\gamma_{n-1}} \left\| \gamma_{n-1} \nabla f(w_{n-1}) - \frac{\gamma_{n-1}}{\beta} \Delta x_{n-1}^1 - \frac{\beta - \gamma_{n-1}}{2\beta - \gamma_{n-1}} \Delta x_{n-1}^2 \right\|^2 \\ & \quad + \frac{\beta\alpha_n^{(2)}}{2} \left\| \nabla f(w_n) - \frac{\beta - \gamma_{n-1}}{\beta} \nabla f(w_{n-1}) - \frac{\gamma_{n-1} - \beta}{\beta^2} \Delta x_{n-1}^1 - \frac{1}{\beta} \Delta x_{n-1}^2 \right\|^2. \end{aligned}$$

The algorithm in subsection 3.1 can be obtained by setting  $\gamma_n = \beta$  for all  $n \geq 0$  and

$$\Delta x_n^1 = \begin{cases} -\beta \Delta x_{n-1} & \text{if } n \text{ is odd,} \\ 0 & \text{if } n \text{ is even,} \end{cases} \quad \Delta x_n^2 = \begin{cases} -\beta \Delta x_n & \text{if } n \text{ is odd,} \\ 0 & \text{if } n \text{ is even} \end{cases}$$

for a sequence  $(\Delta x_n)_{n \geq 0}$  in  $\mathcal{X}$ . Indeed, (3.22) is automatically satisfied for even  $n \geq 0$  and reduces to

$$\|\Delta x_{n-1}\|^2 + \|\Delta x_n\|^2 \leq \alpha_n^{(1)} \|\nabla f(w_{n-1})\|^2 + \alpha_n^{(2)} \|\nabla f(w_n)\|^2$$

for odd  $n$ . This condition is more general than (3.4); however, Theorem 3.1 gives a convergence rate for function values, unlike Theorem 3.4.

*Remark 3.7.* Likewise, if  $f = 0$  on  $\mathcal{X}$ , then (3.11) reads as

$$x_{n+1} := \text{Prox}_g^{\gamma_n} \left( x_n + \frac{\gamma_n}{\beta} \Delta x_n^1 + \Delta x_n^2 \right),$$

and the condition in (3.12) becomes

$$\begin{aligned} & \frac{1}{2\beta} \|\Delta x_n^1\|^2 + \frac{\beta}{2\gamma_n(2\beta - \gamma_n)} \|\Delta x_n^2\|^2 \\ & \leq \alpha_n^{(1)} \frac{2\beta - \gamma_{n-1}}{2\beta\gamma_{n-1}} \left\| x_n - x_{n-1} - \frac{\beta}{2\beta - \gamma_{n-1}} \Delta x_{n-1}^2 \right\|^2 \\ & \quad + \alpha_n^{(2)} \frac{1}{2\beta} \|x_n - x_{n-1} - \Delta x_{n-1}^1\|^2. \end{aligned}$$

Let  $(\Delta x_n)_{n \geq 0}$  be a sequence in  $\mathcal{X}$ , and set  $\Delta x_n^1 = \frac{1}{2} \Delta x_n$ ,  $\Delta x_n^2 = \frac{2\beta - \gamma_n}{2\beta} \Delta x_n$ , and  $\alpha_n^{(2)} = \alpha_n^{(1)}$  for all  $n \geq 0$ ; then the deviation-based proximal point method reads as

$$\begin{aligned} x_{n+1} &= \text{Prox}_g^{\gamma_n}(x_n + \Delta x_n), \\ \|\Delta x_n\|^2 &\leq \alpha_n^{(1)} \frac{4\gamma_n}{\gamma_{n-1}} \left\| x_n - x_{n-1} - \frac{1}{2} \Delta x_{n-1} \right\|^2. \end{aligned}$$

*Remark 3.8.* Setting  $\Delta x_n^1 = \Delta x_n^2 = 0$  recovers the classical forward-backward algorithm [24]. Likewise, the FISTA algorithm [11] can be obtained by setting  $\gamma_n = \beta$  for all  $n \geq 0$ ,

$$\Delta x_n^1 := \frac{t_n - 1}{t_{n+1}} (x_n - x_{n-1}) \quad \text{and} \quad \Delta x_n^2 := 0 \quad \text{for all } n \geq 0,$$

with  $(t_n)_{n=0}^\infty$  defined recursively as  $t_{n+1} := \frac{1}{2}(1 + \sqrt{1 + 4t_n^2})$  for  $n \geq 1$  and  $t_0 := 1$ . This choice satisfies (3.12) with  $\alpha_n^{(1)} = \frac{(t_n-1)^2}{t_{n+1}^2}$  and  $\alpha_n^{(2)} = 0$  for all  $n \geq 1$ .

It is unknown to us if the setting in subsection 3.1 allows us to choose  $\Delta x_n$  that yields an accelerated algorithm. The counterpart of FISTA is Nesterov's accelerated gradient method (5.11), which will in general not yield monotonically nonincreasing function values as guaranteed by Theorem 3.1. Note, however, that there are other accelerated methods with this property; see the discussion in [21, section 4.7.3].

*Remark 3.9.* The usefulness of the condition in (3.12) in comparison to summability conditions in inexact methods can be exemplified by the possibility of choosing  $\Delta x_n^1$  and  $\Delta x_n^2$  parallel with the expressions on the right-hand side, i.e.,

$$\begin{aligned}\Delta x_n^1 &= \nu_{1,n} \sqrt{\frac{2\beta - \gamma_{n-1}}{\gamma_{n-1}}} \left( x_n - x_{n-1} - \frac{\beta}{2\beta - \gamma_{n-1}} \Delta x_{n-1}^2 \right), \\ \Delta x_n^2 &= \nu_{2,n} \sqrt{\gamma_n(2\beta - \gamma_n)} \left( \nabla f(w_n) - \nabla f(w_{n-1}) - \frac{1}{\beta}(x_n - w_{n-1}) \right)\end{aligned}$$

with  $-1 < \nu_{1,n}, \nu_{2,n} < 1$ . The resulting algorithm is a type of momentum method that uses information from previous steps for the current iteration. Generally, arbitrary momentum directions can be used for this approach, as long as they are scaled to satisfy (3.12).

**4. Deep-learning accelerated optimization.** Subsections 3.1 and 3.2 introduced deviation-based iterative schemes for solving smooth and nonsmooth minimization problems that contain many traditional iterative schemes as special cases. These schemes rely on a rule for selecting deviations as iterates progress. Selection rules that give rise to convergent sequences will be referred to as *convergent* and Theorems 3.1 and 3.4 provide criteria that imply convergence.

A natural next step is to look for a selection rule that has “optimal” convergence. This requires one to formalize the notion of “optimal.” Such a selection rule will most likely be problem-dependent; i.e., we cannot expect to have a single selection rule that is optimal for all minimization problems. One approach is to search for the optimization method that is optimal *on average* when applied to a specific class of minimization problems.

Based on the above, we adopt principles from statistical decision theory to learn an optimal convergent selection rule from suitable example data. It is clearly unfeasible to search over all possible convergent selection rules, so attention is restricted to those that are encoded by a (deep) neural network. A key issue is to select a neural network architecture that yields a convergent selection rule. The following sections provide details of this approach.

**4.1. Optimization parametrized by an additional input.** The first task is to specify the class of minimization problems. Here we simply consider minimizing an objective function on  $\mathcal{X}$  that is parametrized by an additional input  $y \in \mathcal{Y}$ , i.e.,

$$(4.1) \quad \min_{x \in \mathcal{X}} F_y(x) \quad \text{for } y \in \mathcal{Y}.$$

$F_y: \mathcal{X} \rightarrow \overline{\mathbb{R}}$  above is a proper, convex, and lower semicontinuous function, and  $\mathcal{Y}$  is the set containing the additional input for the objective, i.e., the input data. For simplicity, we will assume that  $\mathcal{X}$  is finite-dimensional and that each  $F_y$  is strictly convex and coercive so that Corollaries 3.3 and 3.5 are applicable and provide conditions for strong convergence of the respective iteration sequences. Such problems

naturally arise in many applications, e.g., when using variational regularization to solve ill-posed inverse problems (see section 5).

Next is to formalize the notion of *optimal convergence* for an optimization scheme that aims to solve problems of the form in (4.1), which in turn dictates how to train the neural network. Computational feasibility was earlier stated as our main concern, so we consider a setting where the computational budget is limited in terms of a fixed number of iterations  $N$ ; other possible choices are discussed in Remark 4.1. Consider an iterative scheme with parameters  $\theta \in \Theta$  applied for  $N \in \mathbb{N}$  iterations when the initial point  $x_0 \in \mathcal{X}$  is fixed; this defines a function of the input  $y \in \mathcal{Y}$  for the objective:

$$\mathcal{R}_{N,\theta}: \mathcal{Y} \rightarrow \mathcal{X}.$$

Next, the aforementioned optimality means we look for the optimization method, which on average (as  $y \in \mathcal{Y}$  varies) renders an iterate after  $N$  steps that is as close as possible to the true minimal value of the objective in (4.1). Stated mathematically, let  $Y$  denote a  $\mathcal{Y}$ -valued random variable generating  $y \in \mathcal{Y}$ . For optimality we look for  $\mathcal{R}_{N,\hat{\theta}}(\cdot): \mathcal{Y} \rightarrow \mathcal{X}$  where

$$(4.2) \quad \hat{\theta} \in \arg \min_{\theta \in \Theta} \mathbb{E}_Y \left[ F_Y(\mathcal{R}_{N,\theta}(Y)) \right].$$

In practice, the distribution of  $Y$  is not available; instead one has samples  $\{y_i\}_{i=1}^m \subset \mathcal{Y}$  generated by  $Y$ . We therefore consider its empirical counterpart:

$$(4.3) \quad \hat{\theta} \in \arg \min_{\theta \in \Theta} \frac{1}{m} \sum_{i=1}^m F_{y_i}(\mathcal{R}_{N,\theta}(y_i)).$$

The unsupervised learning problem<sup>1</sup> (4.3) results in a scheme that provides an output which is on average closest to minimizing the objective in (4.1) after  $N$  iterations. Combining this with convergence means ensuring the following holds for all  $y \in \mathcal{Y}$ :

$$(4.4) \quad \lim_{n \rightarrow \infty} \mathcal{R}_{n,\hat{\theta}}(y) = x^*(y),$$

where  $x^*(y) \in \mathcal{X}$  solves (4.1). Hence, a solver given by  $\hat{\theta}$  that is trained to be optimal for  $N$  iterates will converge to a solution of (4.1) (as will, in fact, a solver given by *any*  $\theta$ ) if we run more iterations, even when it is not trained for this scenario.

The next two sections provide further details on how to define a deviation-based iterative solver  $\mathcal{R}_{N,\theta}: \mathcal{Y} \rightarrow \mathcal{X}$  in terms of a parametrized selection rule  $\Psi_\theta$  for  $\theta \in \Theta$ . The optimal scheme is obtained by training as in (4.3), and convergence is ensured by an appropriate choice of neural network architecture.

*Remark 4.1.* The topic of how to define “optimal convergence” is of high importance to optimization, but the choice becomes especially apparent in learning-based schemes. In the classical optimization literature [25, 38, 6, 5] the most widely used definition is upper bounds on the worst-case performance as the number of iterations goes to infinity. For learning this is not an attractive definition since it is hard to evaluate computationally. In this work we chose to optimize the average function value after a fixed number of steps, but it would also be possible to optimize, e.g., the average number of steps needed for convergence to some tolerance  $\varepsilon > 0$ .

<sup>1</sup>This is unsupervised learning since training data only consists of  $y_i$ ’s and not the  $x_i^*(y)$ ’s that minimize  $x \mapsto F_{y_i}(x)$ .



**4.2. Smooth convex objective.** Assume that  $F_y: \mathcal{X} \rightarrow \mathbb{R}$  in (4.1) is differentiable with a  $\beta^{-1}$ -Lipschitz continuous gradient. Next, consider iterates  $(x_n(y))_{n \geq 0} \subset \mathcal{X}$  generated by an iterative scheme for solving (4.1). The mapping  $\mathcal{R}_{N,\theta}: \mathcal{Y} \rightarrow \mathcal{X}$  is defined as the  $N$ th term in that sequence, i.e.,  $\mathcal{R}_{N,\theta}(y) = x_N(y)$ .

A natural approach is to consider the deviation-based scheme in (3.3), which, when applied to the  $\mathcal{Y}$ -parametrized minimization in (4.1), reads as

$$x_{n+1}(y) := x_n(y) - \beta \left( \nabla F_y(x_n(y)) + \Delta x_n(y) \right) \quad \text{for all } n \geq 0.$$

The selection rule for setting the deviations  $\Delta x_n(y)$  above is defined as

$$(4.5) \quad \Delta x_n(y) := \Psi_\theta(x_n(y), \nabla F_y(x_n(y)), \Delta x_{n-1}(y)),$$

where  $\Psi_\theta: \mathcal{X} \times \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$  is an updating function that ensures that the resulting deviations satisfy the conditions in (3.4). The above completes the definition of  $\mathcal{R}_{N,x_0}: \mathcal{Y} \times \Theta \rightarrow \mathcal{X}$  and the optimal solver can be obtained from example data by training as in (4.3).

The updating function  $\Psi_\theta$  will be given by a (deep) neural network that is parametrized by  $\theta \in \Theta$ . We construct the neural network  $\Psi_\theta$  by composing a conventional neural network  $\tilde{\Psi}_\theta: \mathcal{X} \times \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$  with a normalizing function  $\mathcal{N}: \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ , which ensures that the outcome is a feasible deviation. More precisely, we define

$$\Psi_\theta(x_n(y), \nabla F_y(x_n(y)), \Delta x_{n-1}(y)) := \mathcal{N}(h_n, \nabla F_y(x_n(y))),$$

where

$$(4.6) \quad \mathcal{N}(h_n, \nabla F_y(x_n(y))) := \alpha \frac{h_n}{\sqrt{\|h_n\|^2 + 1}} \|\nabla F_y(x_n(y))\|,$$

$$(4.7) \quad h_n := \tilde{\Psi}_\theta(x_n(y), \nabla F_y(x_n(y)), \Delta x_{n-1}(y)).$$

Here  $\alpha \in [0, 1]$  controls the size of the learned deviations within the feasible region. Subsection 5.6 presents performance for different values of  $\alpha$  in ablation experiments. Regarding the choice of  $\mathcal{N}$ , the essential property is that its range is contained in the ball that guarantees that the iterative scheme is convergent as formalized in (3.4) with  $\alpha_n = \alpha$ . Thus, one can essentially replace  $\mathcal{N}$  with any transformation with this property.

*Remark 4.2.* In (4.5) we use a current iterate, the gradient at that iterate, and the previous deviation as input for the network  $\Psi_\theta$  that defines the selection rule for the deviations. This choice is largely heuristic and often (as in the problem discussed in subsection 5.2), the objective function naturally consists of two parts  $F_y = f_y + g_y$ . Here one has the option to evaluate gradients of both parts separately instead of taking the gradient of the objective. This corresponds to a selection rule of the form

$$(4.8) \quad \Delta x_n(y) := \Psi_\theta(x_n(y), \nabla f_y(x_n(y)), \nabla g_y(x_n(y)), \Delta x_{n-1}(y))$$

for some neural network  $\Psi_\theta: \mathcal{X} \times \mathcal{X} \times \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ . Moreover, it is possible to incorporate information from previous iterations even though we did not pursue this approach in our experiments.

**4.3. Nonsmooth convex objective.** Here  $F_y: \mathcal{X} \rightarrow \mathbb{R}$  in (4.1) is convex but not necessarily differentiable. We specialize to the case  $F_y = f_y + g_y$ , where  $f_y: \mathcal{X} \rightarrow \mathbb{R}$  and  $g_y: \mathcal{X} \rightarrow \overline{\mathbb{R}}$  are convex but only  $f_y$  is differentiable and  $\nabla f_y: \mathcal{X} \rightarrow \mathcal{X}$  is  $\beta^{-1}$ -Lipschitz continuous. We seek to solve

$$(4.9) \quad \min_{x \in \mathcal{X}} f_y(x) + g_y(x).$$

A natural approach is to consider the deviation-based scheme in (3.11a)–(3.11b) for solving the above problem. Similarly as in subsection 4.2, the selection rules for the updates  $\Delta x_n^1(y)$  and  $\Delta x_n^2(y)$  are given by neural networks  $\Psi^1$  and  $\Psi^2$ . Similarly to the smooth case in subsection 4.2, we first define output of conventional (unnormalized) neural networks  $\tilde{\Psi}^1$  and  $\tilde{\Psi}^2$  as  $h_n^1$  and  $h_n^2$ :

$$(4.10a) \quad h_n^1 := \tilde{\Psi}_{\theta^1}^1(x_n, \nabla f_y(w_{n-1}), \Delta x_{n-1}^1),$$

$$(4.10b) \quad h_n^2 := \tilde{\Psi}_{\theta^2}^2(x_n, \nabla f_y(w_{n-1}), \Delta x_{n-1}^2, \Delta x_n^1).$$

Then, we normalize  $h_n^1$  and  $h_n^2$  so that the resulting learned updates  $\Delta x_n^1$  and  $\Delta x_n^2$  satisfy (3.12) in Theorem 3.4.

$$(4.11a)$$

$$\Delta x_n^1 := \sqrt{\alpha^{(1)} \frac{2\beta - \gamma}{\gamma}} \frac{h_n^1}{\sqrt{\|h_n^1\|^2 + 1}} \left\| x_n - x_{n-1} - \frac{\beta}{2\beta - \gamma} \Delta x_{n-1}^2 \right\|,$$

$$(4.11b)$$

$$\Delta x_n^2 := \sqrt{\alpha^{(2)} \gamma (2\beta - \gamma)} \frac{h_n^2}{\sqrt{\|h_n^2\|^2 + 1}} \left\| \nabla f_y(w_n) - \nabla f_y(w_{n-1}) - \frac{1}{\beta} (x_n - w_{n-1}) \right\|.$$

The normalization step ensures convergence as in (4.4) according to Corollary 3.5. Note that we dropped the dependency on  $y$  in (4.10) and (4.11) to simplify the notation.

The optimal solver is now obtained by training  $\mathcal{R}_{N, x_0}: \mathcal{Y} \times \Theta \rightarrow \mathcal{X}$  as in (4.3). As before,  $\alpha^{(1)}, \alpha^{(2)} \in [0, 1)$  control the sizes of the learned deviations in the feasible region.

**5. Numerical experiments.** This section evaluates performance of the proposed learned optimizers for solving minimization problems arising in tomographic image reconstruction. We selected two optimization problems that have been extensively studied in the research literature. However, it is important to clarify that this paper focuses on optimizing the specified objective function rather than evaluating the quality of the reconstructed images.

**5.1. Variational regularization in tomographic imaging.** Tomography is a collection of techniques that seek to visualize the interior structure of an object by probing it with penetrating particles/waves from different directions. The most well-known example is computed tomography (CT) in medical imaging. A patient is here scanned by X-rays from different directions, and the aim is to recover a 2D/3D image of the interior anatomy.

Tomographic imaging inevitably leads to an inverse problem since the interior structure  $x \in \mathcal{X}$  (2D/3D image) is only indirectly observed through data  $y \in \mathcal{Y}$ . This can be formalized as solving an equation

$$(5.1) \quad y = Ax + \text{noise},$$

where  $A: \mathcal{X} \rightarrow \mathcal{Y}$  models how data is generated in the absence of noise.  $A$  can be taken as a linear operator (ray transform) if one adopts a simplified model for how X-rays interact with tissue and properly preprocesses raw sensor data.

Tomographic imaging problems are unfortunately often ill-posed. This means that a solution procedure that seeks to maximize data consistency, e.g., by minimizing  $x \mapsto \|Ax - y\|^2$ , will be unstable. Variational regularization addresses these issues by adjusting the need for data consistency against the need to suppress unwanted features (over-fitting). Stated formally, instead of trying to solve (5.1), one solves

$$(5.2) \quad \hat{x} \in \arg \min_{x \in \mathcal{X}} \left\{ \|Ax - y\|^2 + S_\lambda(x) \right\}.$$

Here,  $S_\lambda: \mathcal{X} \rightarrow \mathbb{R}$  (regularization functional) stabilizes the recovery procedure, typically by enforcing certain regularity.

*Remark 5.1.* Much effort over the last three decades has been devoted to determining mathematical properties of solutions to (5.2) for various choices of  $S_\lambda$ . Such analysis is typically performed in a nondiscretized setting where both  $x$  (2D/3D image) and  $y$  (data) reside in infinite-dimensional function spaces; see [41, 15, 13] for extensive surveys.

*Characteristics of (5.2).* The minimization in (5.2) is of the form in (4.1) with tomographic data  $y \in \mathcal{Y}$  as the additional variable. In our case, the objective in (5.2) has a natural decomposition  $F_y = f_y + g$  where

$$f_y(x) := \|Ax - y\|^2 \quad \text{and} \quad g(x) := S_\lambda(x).$$

Note here that  $f_y: \mathcal{X} \rightarrow \mathbb{R}$  is differentiable whenever  $A$  is differentiable, which in particular is the case when  $A$  is linear. Properties of  $g: \mathcal{X} \rightarrow \mathbb{R}$  will depend on the choice of regularization functional, but it will be proper, convex, and lower semicontinuous.

Next, in tomographic imaging, both  $x$  and  $y$  are high-dimensional arrays after discretization. In the 2D setting that is considered in subsection 5.6,  $x$  represents  $512 \times 512$  pixel 2D coronal image slices. These are obtained from normal-dose CT scans of the human abdomen provided by the Mayo Clinic for the AAPM Low Dose CT Grand Challenge [34] and used as input for simulating X-ray parallel-beam tomographic data. The latter is from 1000 equidistributed source positions and 1000 detector elements with 5% additive Gaussian noise, i.e., data  $y$  is a  $10^6$ -dimensional array. In summary, even though we consider a 2D setting, (5.2) is a large-scale optimization problem.

An additional aspect to consider is that most clinical imaging studies are performed in a time-critical setting. An informal rule of thumb for CT imaging is that reconstructed images need to be available within 5 minutes after scanning.<sup>2</sup> Hence, applicability of variational regularization in clinical CT rests to a large extent on the ability to (approximately) solve (5.2) within this time-frame, so one can typically only afford about 10 evaluations of  $A$  and  $A^\top$ .

**5.2. Smoothed total-variation regularization.** Total-variation regularization corresponds to choosing  $S_\lambda(x) := \lambda \|\nabla x\|_1$  in (5.2) for some  $\lambda > 0$ . Using such a regularization functional is popular since it preserves edges [40, 16]. When  $x$  is discretized, the gradient  $\nabla$  is typically replaced by the linear operator  $D: \mathcal{X} \rightarrow \mathcal{X} \times \mathcal{X}$  that computes differences in pixel values along vertical and horizontal directions.

<sup>2</sup>This would exclude usages of CT in trauma where reconstructions need to be available instantaneously.

A computational drawback with total variation regularization is that it results in minimizing a nonsmooth objective. A common remedy is therefore to replace the 1-norm with the Huber function, i.e., choosing  $S_\lambda(x) := \lambda H_\delta(Dx)$  where  $\delta > 0$ ,  $x = (x_i)_{i \in I}$ , and

$$(5.3) \quad H_\delta(x) := \sum_{i \in I} h_\delta(x_i), \quad \text{with} \quad h_\delta(x_i) := \begin{cases} \frac{1}{2\delta} x_i^2 & \text{if } |x_i| < \delta, \\ |x_i| - \frac{\delta}{2} & \text{otherwise.} \end{cases}$$

This results in a smooth objective as (5.2) now reads as

$$(5.4) \quad \hat{x} := \arg \min_{x \in \mathcal{X}} \|Ax - y\|^2 + \lambda H_\delta(Dx),$$

which corresponds to (4.1) with  $F_y(x) = f_y(x) + g(x)$  where

$$(5.5) \quad f_y(x) := \|Ax - y\|^2 \quad \text{and} \quad g(x) := \lambda H_\delta(Dx),$$

so

$$(5.6) \quad \nabla F_y(x) = \nabla f_y(x) + \nabla g(x) = 2A^\top(Ax - y) + \lambda D^\top \nabla(H)_\delta(Dx).$$

One may furthermore normalize the linear operator  $A$ , i.e.,  $\|A\| = 1$ . Then,  $\nabla F_y$  is Lipschitz continuous with constant  $\beta^{-1} = 2\|A\|^2 + \frac{\lambda}{\delta}\|D\|^2 = 2 + 8\lambda/\delta$ .

The tests in subsection 5.6 compare various deviation-based schemes for solving (5.4) with  $\delta = 0.01$  (Huber parameter) and  $\lambda = 0.0015$  (regularization parameter). The performance of the learned deviation-based scheme in subsection 4.2 is evaluated for different values of the normalization parameter  $\alpha$  in (4.6).

**5.3. Sparsity-promoting regularization.** Sparsity is another powerful method for regularizing ill-posed inverse problems, and in particular those that are under-sampled [23]. A typical choice as regularizing functional in (5.2) is to pick  $S_\lambda(x) = \lambda\|Wx\|_1$  where  $\lambda > 0$ , so we end up with the following nonsmooth minimization problem:

$$(5.7) \quad \hat{x} := \arg \min_x \|Ax - y\|^2 + \lambda\|Wx\|_1.$$

In the above,  $x \mapsto Wx$  is a compression that preserves essential features (sparsifying transform). For the tests in subsection 5.6, we select it as an orthogonal wavelet transform given by Symlets with filter length 10 and 5 scale levels. The regularization parameter is set to  $\lambda = 0.0005$ .

The minimization in (5.7) corresponds to (4.1) with a nonsmooth objective  $F_y(x) = f_y(x) + g(x)$  where

$$(5.8) \quad f_y(x) := \|Ax - y\|^2 \quad \text{and} \quad g(x) := \|Wx\|_1.$$

As in subsection 5.2,  $A$  is normalized, so  $\|A\| = 1$ , and  $\nabla f_y(x) = 2A^\top(Ax - y)$  is Lipschitz continuous with constant  $\beta^{-1} = 2$ . Hence,  $f_y$  and  $g$  satisfy the assumptions in subsection 3.2, and one can solve the problem by using the method proposed in subsection 4.3. To proceed, we use [10, Proposition 24.14] to compute the proximal in (3.11b):

$$(5.9) \quad \text{Prox}_g^{\lambda\gamma_n}(x) = x + \frac{1}{\mu} W^\top \left( \text{Prox}_{\|\cdot\|_1}^{\mu\lambda\gamma_n}(Wx) - Wx \right),$$

where  $\text{Prox}_{\|\cdot\|_1}^\gamma(x) = \text{sign}(x) \cdot \max(|x| - \gamma, 0)$  is a proximal of  $l_1$  norm and  $\mu$  is such that  $\mu \text{Id} = WW^\top$ .

The tests in subsection 5.6 compare various deviation-based schemes for solving (5.7) with gradient descent step-size  $\gamma_n = 1/(2\|A\|^2) = 0.5$ . The performance of the learned deviation-based scheme in subsection 4.3 is evaluated for different values of the normalization parameters  $\alpha^{(1)} = \alpha^{(2)} = \alpha$  in (4.11).

**5.4. Baseline methods.** In the case of smooth optimization (i.e., solving (5.4)), we compare our method to the gradient descent and Nesterov's accelerated scheme [36]. For the gradient descent, the updates are the same as in (3.3), but with  $\Delta x_n = 0$ :

$$(5.10) \quad x_{n+1} := x_n - \beta \nabla f(x_n).$$

In Nesterov's method the updates are

$$(5.11a) \quad x_n := w_n - \beta \nabla f(w_n),$$

$$(5.11b) \quad t_{n+1} := \frac{1 + \sqrt{(1 + 4t_n^2)}}{2},$$

$$(5.11c) \quad w_{n+1} := x_n + \frac{t_n - 1}{t_{n+1}}(x_n - x_{n-1}),$$

where  $t_0 := 1$ .

In the nonsmooth case (i.e., solving (5.7)), we compare our method against ISTA [24] and FISTA [11]. Iterates in ISTA are updated according to

$$(5.12) \quad x_{n+1} := \text{Prox}_g^{\gamma_n}(x_n - \gamma_n \nabla f(x_n)).$$

This is equivalent to updates in (3.11a)–(3.11b) with  $\Delta x_n^1 = \Delta x_n^2 = 0$ . In FISTA, updates are given as

$$(5.13a) \quad x_n := \text{Prox}_g^{\gamma_n}(w_n - \gamma_n \nabla f(w_n)),$$

$$(5.13b) \quad t_{n+1} := \frac{1 + \sqrt{(1 + 4t_n^2)}}{2},$$

$$(5.13c) \quad w_{n+1} := x_n + \frac{t_n - 1}{t_{n+1}}(x_n - x_{n-1}),$$

where  $t_0 := 1$ . Finally, we do not provide a comparison to LISTA [26], because an input of dimension  $n$  requires training  $n^2$  parameters. The memory requirements for this method are prohibitively large for the examples in subsection 5.6 since  $n = 512 \times 512$ .

**5.5. Implementation.** The architecture for the neural networks in (4.5) and (4.10) is similar to that for the networks representing the learned updates in [4]. Thus, it consists of three convolutional layers (two hidden layers and one output layer) with  $3 \times 3$  convolutional kernels. The hidden layers have 32 filters that are followed by instance normalization and leaky ReLU as an activation function. Instead of performing explicit preprocessing of the input data, we use additional instance normalization before the hidden layers. This results in a network with a total of  $1 \cdot 10 \cdot 32 + 32 \cdot 10 \cdot 32 + 32 \cdot 10 \cdot 1 = 10880$  free parameters that need to be set during training.

To train the networks, we use the unsupervised loss function  $F(x_N)$ , where  $N$  is the total number of iterations. However, during the evaluation, we run many more iterations to verify the convergence, despite the fact that the network was not trained for this scenario. During the training, we sample  $N$  uniformly between 10 and 20.

Randomizing the number of iterations is necessary to ensure a smooth transition between the scenario for which the model is trained and the scenario for which it is not. Since the loss function  $F(x_n)$  is not guaranteed to decrease monotonically, we observed that fixing the number of iterations to 10 during the training results in a slight increase of the loss after 10 iterations, before it starts to decrease again.

The neural networks are implemented in Tensorflow [1] and trained for  $10^5$  iterations using the Adam optimizer with learning rate  $10^{-3}$  and batch size 1 (using a larger batch size is not possible due to the high dimension of the input images and GPU memory constraints). This takes about three days on a workstation with a single GeForce RTX 2080 Ti GPU. However, for most presented models,  $3 \cdot 10^4$  training iterations are sufficient. Tomography-related operations necessary for evaluating the data consistency along with bindings to TensorFlow are implemented in ODL [2] with ASTRA [42] as the computational back-end. We make the code publicly available.<sup>3</sup>

**5.6. Results.** All the solution algorithms initialize iterations to zero image  $x_0 = 0$ . Next, all deviation-based schemes with learned selection rules have been trained on data from nine of ten patients. The 210 images corresponding to the last patient are used for testing.

#### 5.6.1. Smoothed total variation regularization (smooth optimization).

Figure 5.1 visualizes the performance of different methods averaged over the set of test optimization problems (5.4). For each input datum  $y$  we identify the minimal objective value  $F_y^*$  achieved among all the methods. Then, we compute the relative difference  $(F_y(x_n) - F_y^*)/F_y^*$  at each iteration  $n$ . As the last step we average those values over the test cases and plot them depending on the iteration in logarithmic scale.

We present results for the following methods: the gradient descent, Nesterov's accelerated method, and the learned scheme described in subsection 4.2 with the normalization parameter  $\alpha = 0.9$ . For comparison, we also include a variant of the learned scheme without the normalization given by (4.6).

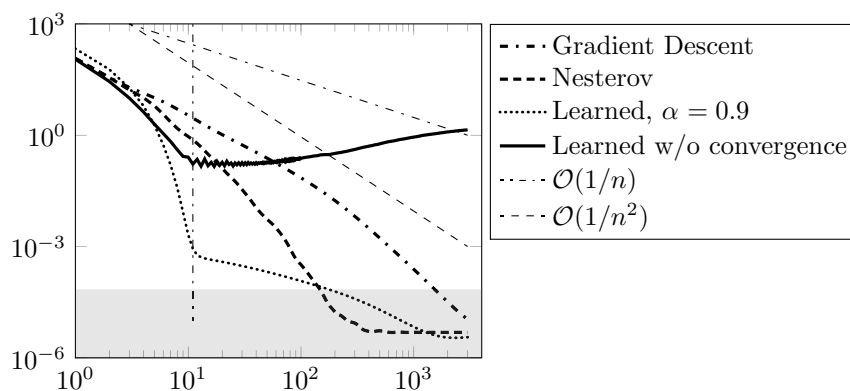


FIG. 5.1. Plot of  $(F_y(x_n) - F_y^*)/F_y^*$  averaged over the test cases as a function of the iteration number  $n$  for smooth optimization methods: The gradient descent, Nesterov's accelerated method, the learned scheme in subsection 4.2 with  $\alpha = 0.9$ , and a variant of latter without the normalization given by (4.6). Values below numerical precision (based on the monotonicity of the gradient descent) are shaded in gray.

<sup>3</sup>[github.com/JevgenijaAksjonova/Deep-Optimization](https://github.com/JevgenijaAksjonova/Deep-Optimization)

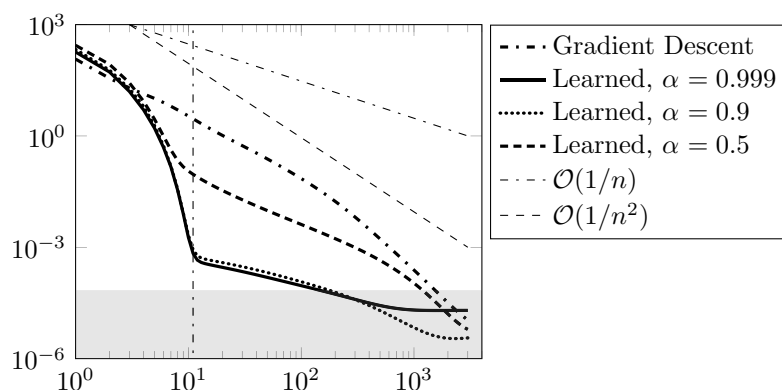


FIG. 5.2. Plot of  $(F_y(x_n) - F_y^*)/F_y^*$  averaged over the test cases as a function of the iteration number  $n$  for smooth optimization methods: The gradient descent and the learned scheme in subsection 4.2 that enforces convergence through (4.6) for different values of  $\alpha$ . Values below numerical precision (based on the monotonicity of the gradient descent) are shaded in gray.

The level of numerical precision in our tests is mostly dictated by numerical precision of implementation of the operator  $A$  and its adjoint. These are implemented in the tomographic package ASTRA [42] using 32-bit single-precision floating-point numbers (that represents about 7 decimal digits). Moreover, the machine precision is not guaranteed by the package. To estimate the level of numerical precision that can be practically reached, we use a theoretical fact that the gradient descent method is monotonically decreasing. We identify the average relative error at which this method stops decreasing monotonically ( $\approx 7 \cdot 10^{-5}$ ) and shadow the region below this level in Figures 5.1 and 5.2. Although all the methods continue to decrease below this level, any observations must be analyzed with caution.

As we can see, learned schemes perform better than the baseline methods during the first 10 iterations (marked with a grey vertical line). This result is expected since it corresponds to the objective set during the training. We also evaluate a variant of the learned scheme that uses a network without the normalization in (4.6) and, hence, does not have convergence guarantees. Not only does it achieve a worse performance during the first 10 iterations, but it also quickly diverges afterwards.

In Figure 5.2, we evaluate the performance of the learned scheme for different choices of the normalization parameter  $\alpha$ , which controls the size of the learned deviation  $\Delta x_n$ . The largest value  $\alpha = 0.999$  leads to the best performance during the first 10 iterations; however,  $\alpha = 0.9$  performs almost as well and converges much faster. Decreasing  $\alpha$  further makes the method approach the gradient descent, by enforcing convergence and sacrificing performance during the first 10 iterations.

### 5.6.2. Sparsity-promoting regularization (nonsmooth optimization).

Figure 5.3 visualizes the performance of different methods averaged over the set of test optimization problems (5.7). For each input  $y$  we identify the minimal objective value  $F_y^*$  achieved among all the methods. Then, we compute the relative difference  $(F_y(x_n) - F_y^*)/F_y^*$  at each iteration  $n$ . As the last step we average those values over the test cases and plot them depending on the iteration in logarithmic scale.

As in the smooth optimization experiments, we estimate the level of numerical precision that can be practically reached, using monotonicity of ISTA. We identify the average relative error at which this method stops decreasing monotonically

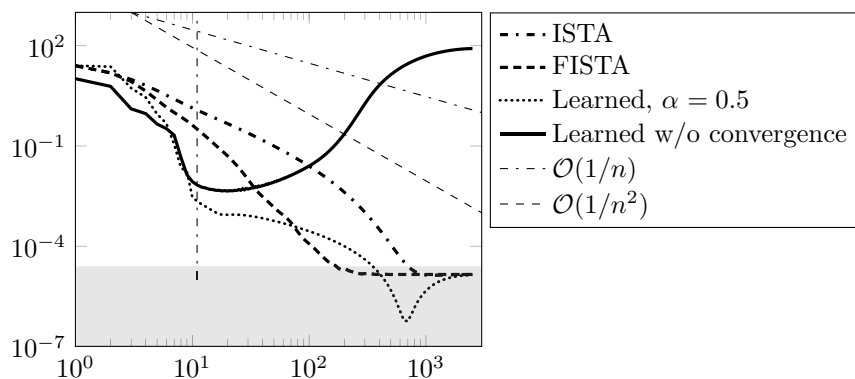


FIG. 5.3. Plot of  $(F_y(x_n) - F_y^*)/F_y^*$  averaged over the test cases as a function of the iteration number  $n$  for nonsmooth optimization methods: ISTA [24], FISTA [11], the learned scheme in subsection 4.3 with  $\alpha = 0.5$ , and a variant of latter without the normalization given by (4.11). Values below numerical precision (based on the monotonicity of ISTA) are shaded in gray.

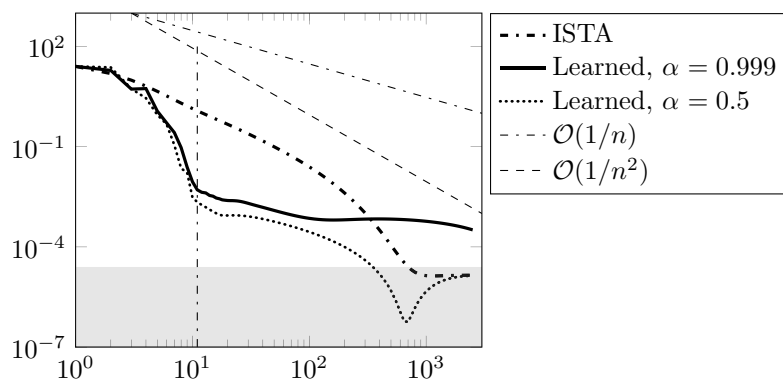


FIG. 5.4. Plot of  $(F_y(x_n) - F_y^*)/F_y^*$  averaged over the test cases as a function of the iteration number  $n$  for nonsmooth optimization methods: ISTA [24] and the learned scheme in subsection 4.3 with convergent updates by (4.11) for different values of  $\alpha$ . Values below numerical precision (based on the monotonicity of ISTA) are shaded in gray.

( $\approx 2.5 \times 10^{-5}$ ) and shadow the region below this level in Figures 5.3 and 5.4. We can see that the convergent methods converge to the same objective value, even though the learned method is able to achieve a numerically smaller objective before that.

We can see that, during the first 10 iterations (marked with a grey vertical line), the learned methods perform significantly better than the two baseline methods: ISTA [24] and FISTA [11]. Moreover, the normalization step (4.11) not only ensures convergence; it also improves the performance after 10 iterations.

Next, we show the effect of choosing different normalization parameters  $\alpha$  in Figure 5.4. As in smooth optimization, decreasing  $\alpha$  improves convergence. Moreover, in this case  $\alpha = 0.5$  leads to the best performance among the tested methods during the first 10 iterations as well.

**6. Discussion.** The figures in subsection 5.6 illustrate that the learned methods with convergence guarantees perform very well for the initial 10 iterations, as they were trained to do. However, beyond that point, their convergence becomes notice-



ably slower compared to the baseline methods. In fact, using a learned scheme for the first 10 iterations and then switching to a convergent scheme like Nesterov's scheme or FISTA could lead to an overall faster convergence. The purpose of showcasing the performance of learned methods until convergence is to support theoretical claims in subsections 3.1 and 3.2. However, the practical benefit is achieved in the setting where these methods were trained (during the first 10 iterations). In particular, the main advantage of the convergent methods is that they are guaranteed to work even on out-of-distribution data. This provides additional robustness, which is supported by our empirical observation in subsection 5.6 that learned schemes with convergence guarantees outperform the analogous schemes without convergence guarantees on the test set.

In our experiments, we found that evaluating a single convolutional neural network (CNN) for computing deviation  $\Delta x_n(y)$  (as in (4.5)) takes approximately the same time as computing a gradient of the smooth objective in (5.4). Specifically, the gradient descent takes about 0.075s per iteration, while the learned scheme in subsection 4.2 takes 0.166s. However, CNNs are known to be highly parallelizable and can be efficiently executed on GPUs. This often cannot be said about the forward operator  $A$  in the inverse problem (5.1). Our proposed method will be advantageous in scenarios where computational costs are dominated by the evaluation of the forward operator rather than CNNs. We expect this to be true in many practical applications, in particular 3D CT.

**7. Conclusion.** Our goal was to construct deep-learning accelerated algorithms for convex optimization, and we began by noting that these heavily over-parametrized algorithms do not fit into classical optimization schemes, which only use a few parameters.

To do this, we proved convergence of two convex optimization algorithms, one for smooth optimization and one for nonsmooth. Instead of showing their convergence under some specific parameter choice, we showed convergence whenever the updates are chosen in particular sets of full dimension. Given that these sets are rather large, there is hence an enormous number of options to choose from. To make the best choice in any situation we used a neural network and trained it to pick the updates for optimal convergence speed.

We applied our algorithms to the problem of regularized image reconstruction in CT and showed that our algorithm outperforms classical algorithms, especially when only a few iterations are applied. Furthermore, our algorithm converged to the exact solution, as expected.

We hope that this work will open new avenues in combining deep learning with provable convergence, and we see several interesting open problems arising from it. For one, our algorithms are only applicable to a subset of all convex problems and it would be particularly interesting to find a primal-dual scheme which can be combined with deep learning. Another interesting research direction is to investigate other definitions of *the best* optimization algorithm; for example, one highly sought-after feature is the ability to train our network to converge to a given accuracy as quickly as possible. Finally, one could combine the deterministic acceleration in the spirit of Nesterov [36] or [11] with the deep learning techniques introduced in this paper and find an optimal trade-off between the two.

**Acknowledgments.** We would like to thank Pooria Joulani and Pushmeet Kohli for reviewing and helping us improve the manuscript and Pontus Giselsson for fruitful discussions. This work was partially done while the first author was at KTH and while the fourth author was at KTH and Elekta.

## REFERENCES

- [1] M. ABADI, A. AGARWAL, P. BARHAM, E. BREVDO, Z. CHEN, C. CITRO, G. S. CORRADO, A. DAVIS, J. DEAN, M. DEVIN, S. GHEMAWAT, I. GOODFELLOW, A. HARP, G. IRVING, M. ISARD, Y. JIA, R. JOZEFOWICZ, L. KAISER, M. KUDLUR, J. LEVENBERG, D. MANÉ, R. MONGA, S. MOORE, D. MURRAY, C. OLAH, M. SCHUSTER, J. SHLENS, B. STEINER, I. SUTSKEVER, K. TALWAR, P. TUCKER, V. VANHOUCKE, V. VASUDEVAN, F. VIÉGAS, O. VINYALS, P. WARDEN, M. WATTENBERG, M. WICKE, Y. YU, AND X. ZHENG, *TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*, <https://arxiv.org/abs/1603.04467v2>, 2016.
- [2] J. ADLER, H. KOHR, AND O. ÖKTEM, *Operator Discretization Library (ODL)*, <https://github.com/odlgroup/odl/>, 2017.
- [3] J. ADLER AND O. ÖKTEM, *Solving ill-posed inverse problems using iterative deep neural networks*, *Inverse Problems*, 33 (2017), 124007, <https://doi.org/10.1088/1361-6420/aa9581>.
- [4] J. ADLER AND O. ÖKTEM, *Learned primal-dual reconstruction*, *IEEE Trans. Med. Imaging*, 37 (2018), pp. 1322–1332, <https://doi.org/10.1109/TMI.2018.2799231>.
- [5] M. AHOOKHOSH AND A. NEUMAIER, *An optimal subgradient algorithm for large-scale bound-constrained convex optimization*, *Math. Methods Oper. Res.*, 86 (2017), pp. 123–147, <https://doi.org/10.1007/s00186-017-0585-1>.
- [6] M. AHOOKHOSH AND A. NEUMAIER, *Optimal subgradient algorithms for large-scale convex optimization in simple domains*, *Numer. Algorithms*, 76 (2017), pp. 1071–1097, <https://doi.org/10.1007/s11075-017-0297-x>.
- [7] M. ANDRYCHOWICZ, M. DENIL, S. GOMEZ, M. W. HOFFMAN, D. PFAU, T. SCHAUL, B. SHILLINGFORD, AND N. DE FREITAS, *Learning to learn by gradient descent by gradient descent*, in *Advances in Neural Information Processing Systems*, Vol. 29, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, eds., 2016, <https://papers.nips.cc/paper/2016/hash/fb87582825f9d28a8d42c5e5e8b23d-Abstract.html>.
- [8] S. ARRIDGE, P. MAASS, O. ÖKTEM, AND C.-B. SCHÖNLIEB, *Solving inverse problems using data-driven models*, *Acta Numer.*, 28 (2019), pp. 1–174, <https://doi.org/10.1017/S0962492919000059>.
- [9] S. BANERT, A. RINGH, J. ADLER, J. KARLSSON, AND O. ÖKTEM, *Data-driven nonsmooth optimization*, *SIAM J. Optim.*, 30 (2020), pp. 102–131, <https://doi.org/10.1137/18M1207685>.
- [10] H. H. BAUSCHKE AND P. L. COMBETTES, *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*, 2nd ed., CMS Books in Mathematics 408, Springer, Cham, 2017, <https://doi.org/10.1007/978-3-319-48311-5>.
- [11] A. BECK AND M. TEOULLE, *A fast iterative shrinkage-thresholding algorithm for linear inverse problems*, *SIAM J. Imaging Sci.*, 2 (2009), pp. 183–202, <https://doi.org/10.1137/080716542>.
- [12] I. BELLO, B. ZOPH, V. VASUDEVAN, AND Q. V. LE, *Neural optimizer search with reinforcement learning*, in *Proceedings of the 34th International Conference on Machine Learning*, PMLR, 2017, pp. 459–468, <https://proceedings.mlr.press/v70/bello17a.html>.
- [13] M. BENNING AND M. BURGER, *Modern regularization methods for inverse problems*, *Acta Numer.*, 27 (2018), pp. 1–111, <https://doi.org/10.1017/S0962492918000016>.
- [14] L. M. BRICEÑO-ARIAS AND P. L. COMBETTES, *A monotone + skew splitting model for composite monotone inclusions in duality*, *SIAM J. Optim.*, 21 (2011), pp. 1230–1250, <https://doi.org/10.1137/10081602X>.
- [15] M. BURGER AND S. OSHER, *A guide to the TV zoo*, in *Level Set and PDE Based Reconstruction Methods in Imaging*, Springer, Cham, 2013, pp. 1–70, [https://doi.org/10.1007/978-3-319-01712-9\\_1](https://doi.org/10.1007/978-3-319-01712-9_1).
- [16] V. CASELLES, A. CHAMBOLLE, AND M. NOVAGA, *Total variation in imaging*, in *Handbook of Mathematical Methods in Imaging*, 2nd ed., Springer New York, 2015, pp. 1455–1499, [https://doi.org/10.1007/978-1-4939-0790-8\\_23](https://doi.org/10.1007/978-1-4939-0790-8_23).
- [17] A. CAUCHY, *Méthode générale pour la résolution des systemes d'équations simultanées*, *C. R. Acad. Sci.*, 25 (1847), pp. 536–538, <https://gallica.bnf.fr/ark:/12148/bpt6k2982c/f540.item>.
- [18] T. CHEN, X. CHEN, W. CHEN, H. HEATON, J. LIU, Z. WANG, AND W. YIN, *Learning to optimize: A primer and a benchmark*, *J. Mach. Learn. Res.*, 23 (2022), pp. 1–59, <http://jmlr.org/papers/v23/21-0308.html>.
- [19] X. CHEN, J. LIU, Z. WANG, AND W. YIN, *Theoretical linear convergence of unfolded ISTA and its practical weights and thresholds*, in *Advances in Neural Information Processing Systems*, Vol. 31, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, eds., 2018, <https://proceedings.neurips.cc/paper/2018/hash/cf8c9be2a4508a24ae92c9d3d379131d-Abstract.html>.

- [20] P. L. COMBETTES AND J.-C. PESQUET, *Primal-dual splitting algorithm for solving inclusions with mixtures of composite, Lipschitzian, and parallel-sum type monotone operators*, Set-Valued Var. Anal., 20 (2012), pp. 307–330, <https://doi.org/10.1007/s11228-011-0191-y>.
- [21] A. D'ASPREMONT, D. SCIEUR, A. TAYLOR, ET AL., *Acceleration methods*, Found. Trends Optim., 5 (2021), pp. 1–245.
- [22] Y. DRORI AND M. TEBoulLE, *Performance of first-order methods for smooth convex minimization: A novel approach*, Math. Program., 145 (2014), pp. 451–482, <https://doi.org/10.1007/s10107-013-0653-0>.
- [23] M. FORNASIER AND H. RAUHUT, *Compressive sensing*, in Handbook of Mathematical Methods in Imaging, 2nd ed., Springer, New York, 2015, pp. 205–256, [https://doi.org/10.1007/978-1-4939-0790-8\\_6](https://doi.org/10.1007/978-1-4939-0790-8_6).
- [24] A. A. GOLDSTEIN, *Convex programming in Hilbert space*, Bull. Amer. Math. Soc., 70 (1964), pp. 709–710, <https://doi.org/10.1090/S0002-9904-1964-11178-2>.
- [25] C. C. GONZAGA, E. W. KARAS, AND D. R. ROSSETTO, *An optimal algorithm for constrained differentiable convex optimization*, SIAM J. Optim., 23 (2013), pp. 1939–1955, <https://doi.org/10.1137/110836602>.
- [26] K. GREGOR AND Y. LECUN, *Learning fast approximations of sparse coding*, in Proceedings of the 27th International Conference on Machine Learning (ICML 2010), 2010, pp. 399–406, <https://icml.cc/Conferences/2010/papers/449.pdf>.
- [27] K. HAMMERNIK, T. KLATZER, E. KOBLER, M. P. RECHT, D. K. SODICKSON, T. POCK, AND F. KNOLL, *Learning a variational network for reconstruction of accelerated MRI data*, Magn. Reson. Med., 79 (2018), pp. 3055–3071, <https://doi.org/10.1002/mrm.26977>.
- [28] H. HEATON, X. CHEN, Z. WANG, AND W. YIN, *Safeguarded learned convex optimization*, Proc. AAAI Conf. Artif. Intell., 37 (2023), pp. 7848–7855.
- [29] J.-T. HSIEH, S. ZHAO, S. EISMANN, L. MIRABELLA, AND S. ERMON, *Learning neural PDE solvers with convergence guarantees*, in International Conference on Learning Representations (ICLR 2019), 2019, <https://openreview.net/forum?id=rklaWn0qK7>.
- [30] K. LI AND J. MALIK, *Learning to optimize*, in International Conference on Learning Representations, 2016.
- [31] J. LIANG, T. LUO, AND C.-B. SCHÖNLIEB, *Improving “fast iterative shrinkage-thresholding algorithm”: Faster, smarter, and greedier*, SIAM J. Sci. Comput., 44 (2022), pp. A1069–A1091, <https://doi.org/10.1137/21M1395685>.
- [32] R. LIU, S. CHENG, Y. HE, X. FAN, Z. LIN, AND Z. LUO, *On the convergence of learning-based iterative methods for nonconvex inverse problems*, IEEE Trans. Pattern Anal. Mach. Intell., 42 (2020), pp. 3027–3039, <https://doi.org/10.1109/TPAMI.2019.2920591>.
- [33] N. MAHESWARANATHAN, D. SUSSILLO, L. METZ, R. SUN, AND J. SOHL-DICKSTEIN, *Reverse engineering learned optimizers reveals known and novel mechanisms*, in Advances in Neural Information Processing Systems, Vol. 34, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, eds., Curran Associates, 2021, pp. 19910–19922, <https://proceedings.neurips.cc/paper/2021/file/a57ecd54d4df7d999bd9c5e3b973ec75-Paper.pdf>.
- [34] C. H. MCCOLLOUGH, A. C. BARTLEY, R. E. CARTER, B. CHEN, T. A. DREES, P. EDWARDS, D. R. HOLMES, A. E. HUANG, F. KHAN, S. LENG, K. L. McMILLAN, G. J. MICHALAK, K. M. NUNEZ, L. YU, AND J. G. FLETCHER, *Low-dose CT for the detection and classification of metastatic liver lesions: Results of the 2016 Low Dose CT Grand Challenge*, Med. Phys., 44 (2017), pp. e339–e352, <https://doi.org/10.1002/mp.12345>.
- [35] L. METZ, N. MAHESWARANATHAN, C. D. FREEMAN, B. POOLE, AND J. SOHL-DICKSTEIN, *Tasks, Stability, Architecture, and Compute: Training More Effective Learned Optimizers, and Using Them to Train Themselves*, <https://arxiv.org/abs/2009.11243v1>, 2020.
- [36] Y. E. NESTEROV, *A method of solving a convex programming problem with convergence rate  $O(1/k^2)$* , Sov. Math. Dokl., 27 (1983), pp. 372–376.
- [37] Y. E. NESTEROV, *Introductory Lectures on Convex Optimization. A Basic Course*, Appl. Optim., 87, Springer, New York, 2004, <https://doi.org/10.1007/978-1-4419-8853-9>.
- [38] A. NEUMAIER, *OSGA: A fast subgradient algorithm with optimal complexity*, Math. Program., 158 (2016), pp. 1–21, <https://doi.org/10.1007/s10107-015-0911-4>.
- [39] R. T. ROCKAFELLAR, *Convex Analysis*, Princeton Math. Ser. 36, Princeton University Press, Princeton, NJ, 1970, <https://doi.org/10.1515/9781400873173>.
- [40] L. I. RUDIN, S. OSHER, AND E. FATEMI, *Nonlinear total variation based noise removal algorithms*, Phys. D, 60 (1992), pp. 259–268, [https://doi.org/10.1016/0167-2789\(92\)90242-F](https://doi.org/10.1016/0167-2789(92)90242-F).
- [41] O. SCHERZER, M. GRASMAIR, H. GROSSAUER, M. HALTMEIER, AND F. LENZEN, *Variational Methods in Imaging*, Appl. Math. Sci. 167, Springer, New York, 2009, <https://doi.org/10.1007/978-0-387-69277-7>.

- [42] W. VAN AARLE, W. J. PALENSTIJN, J. CANT, E. JANSSENS, F. BLEICHRODT, A. DABRAVOLSKI, J. DE BEENHOUWER, K. J. BATENBURG, AND J. SIJBERS, *Fast and flexible X-ray tomography using the ASTRA toolbox*, Opt. Express, 24 (2016), pp. 25129–25147, <https://doi.org/10.1364/OE.24.025129>.
- [43] B. C. VŨ, *A splitting algorithm for dual monotone inclusions involving cocoercive operators*, Adv. Comput. Math., 38 (2013), pp. 667–681, <https://doi.org/10.1007/s10444-011-9254-8>.
- [44] O. WICHROWSKA, N. MAHESWARANATHAN, M. W. HOFFMAN, S. G. COLMENAREJO, M. DENIL, N. FREITAS, AND J. SOHL-DICKSTEIN, *Learned optimizers that scale and generalize*, in International Conference on Machine Learning, PMLR, 2017, pp. 3751–3760, <https://proceedings.mlr.press/v70/wichrowska17a.html>.
- [45] Y. YANG, J. SUN, H. LI, AND Z. XU, *Deep ADMM-net for compressive sensing MRI*, in Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS’16), 2016, pp. 10–18, <https://papers.nips.cc/paper/2016/hash/1679091c5a880faf6fb5e6087eb1b2dc-Abstract.html>.